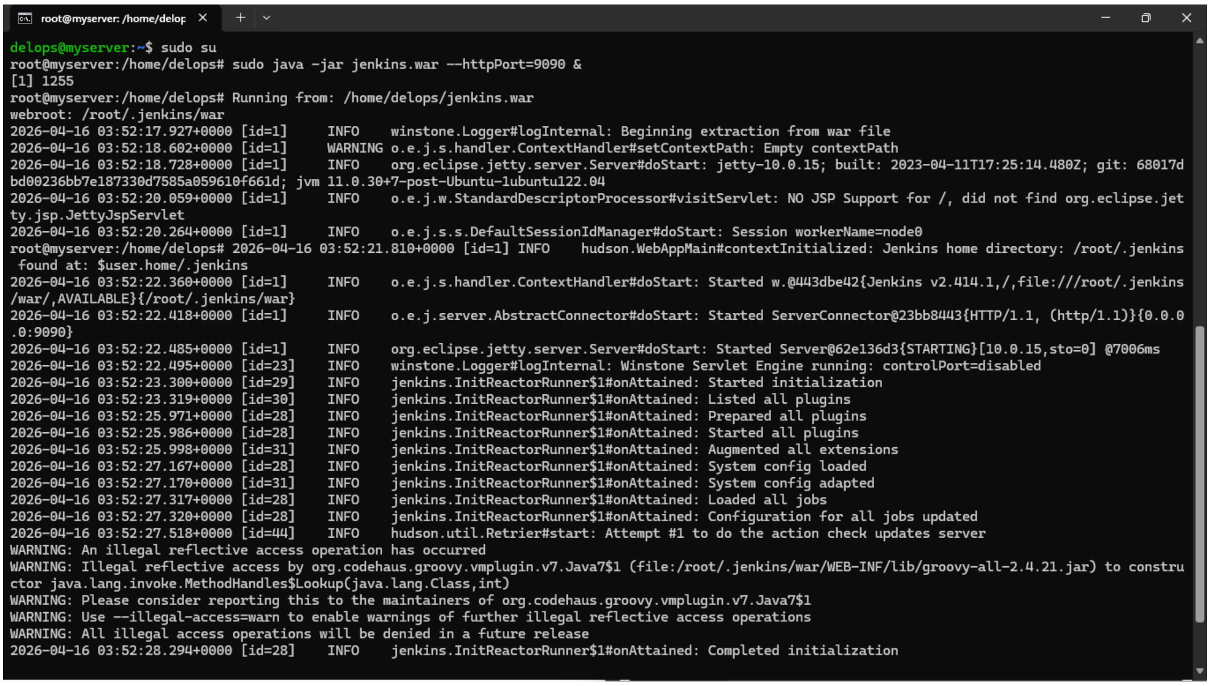
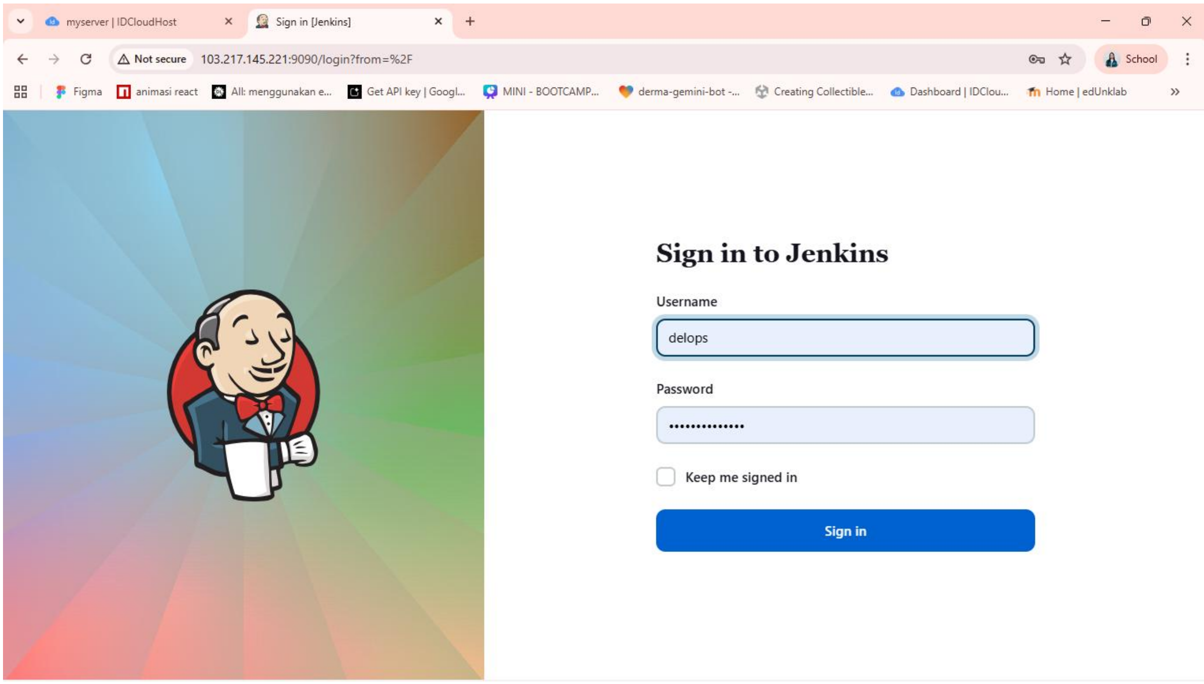
DevOps – Report CI/CD

Nama: Deeva Civia Aulia Lolong & Wendel Mnahonin

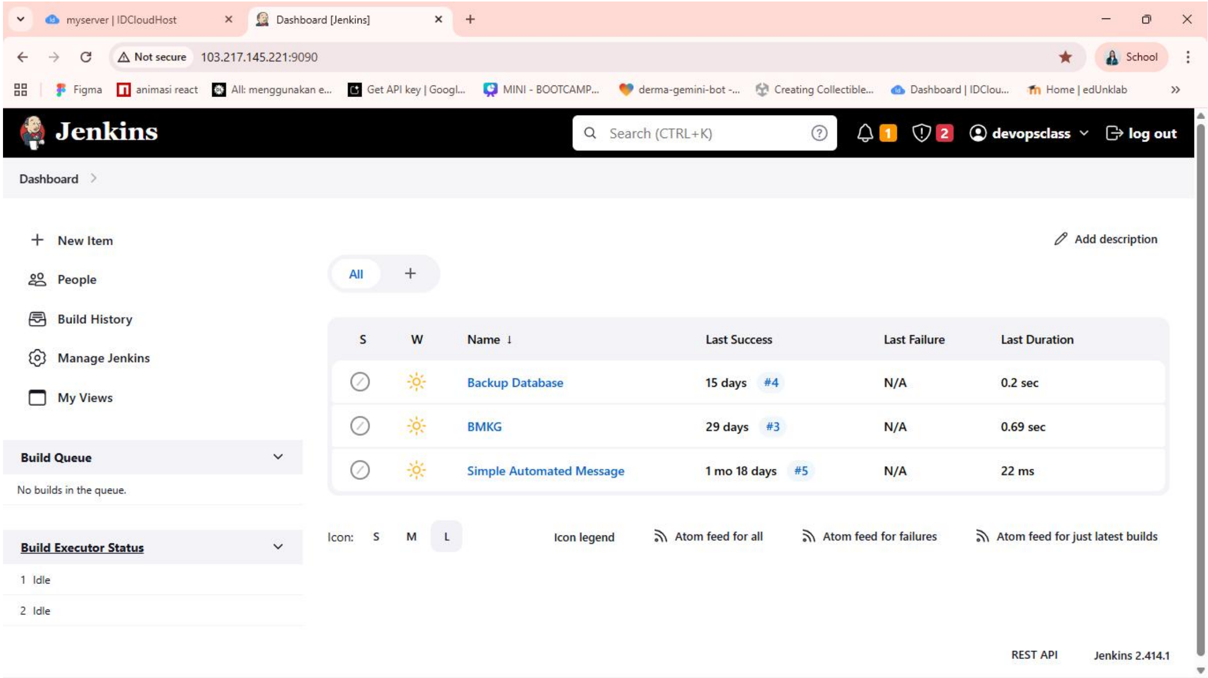
1. Login ke Server dan aktifkan Jenkins



2. Login ke dalam Jenkins

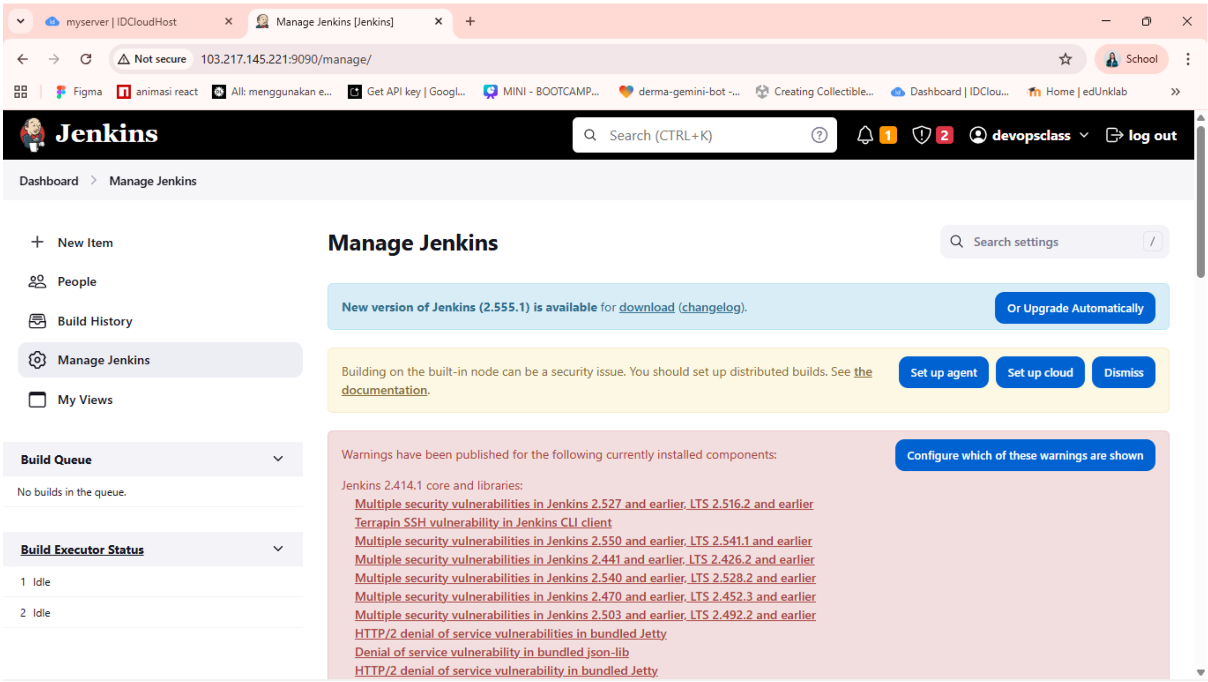


Tampilan Dashboard Jenkins

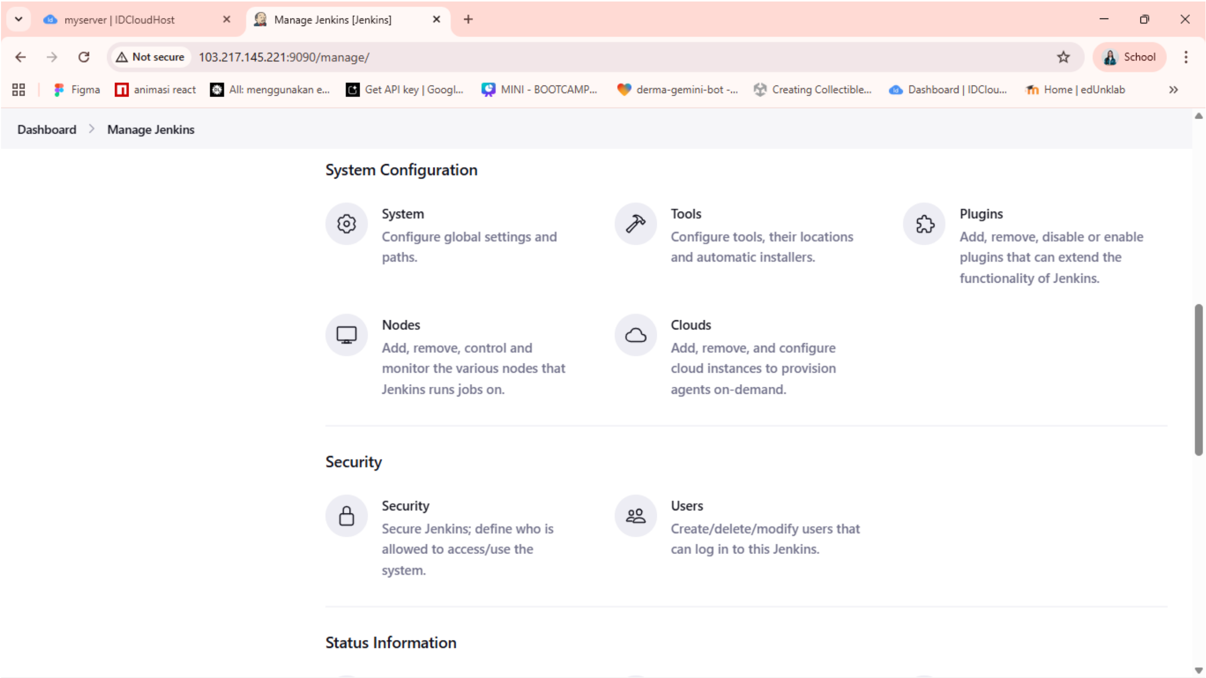


3. Install Git Plugin

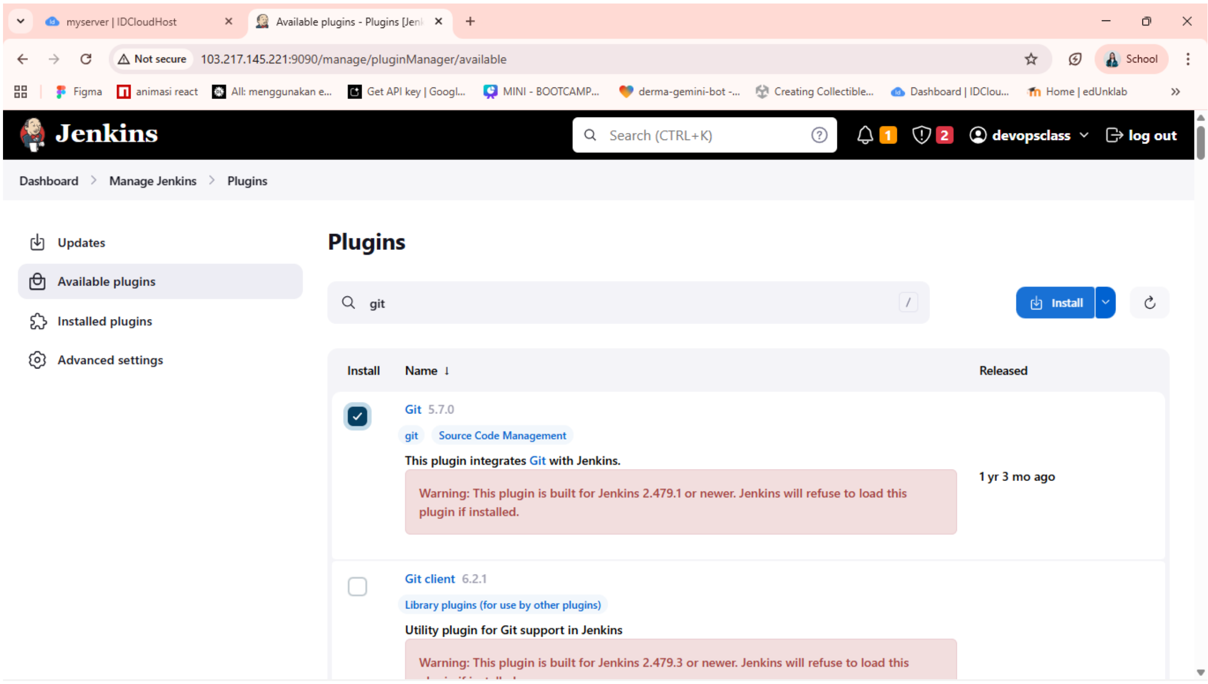
Buka menu “Menage Jenkins”

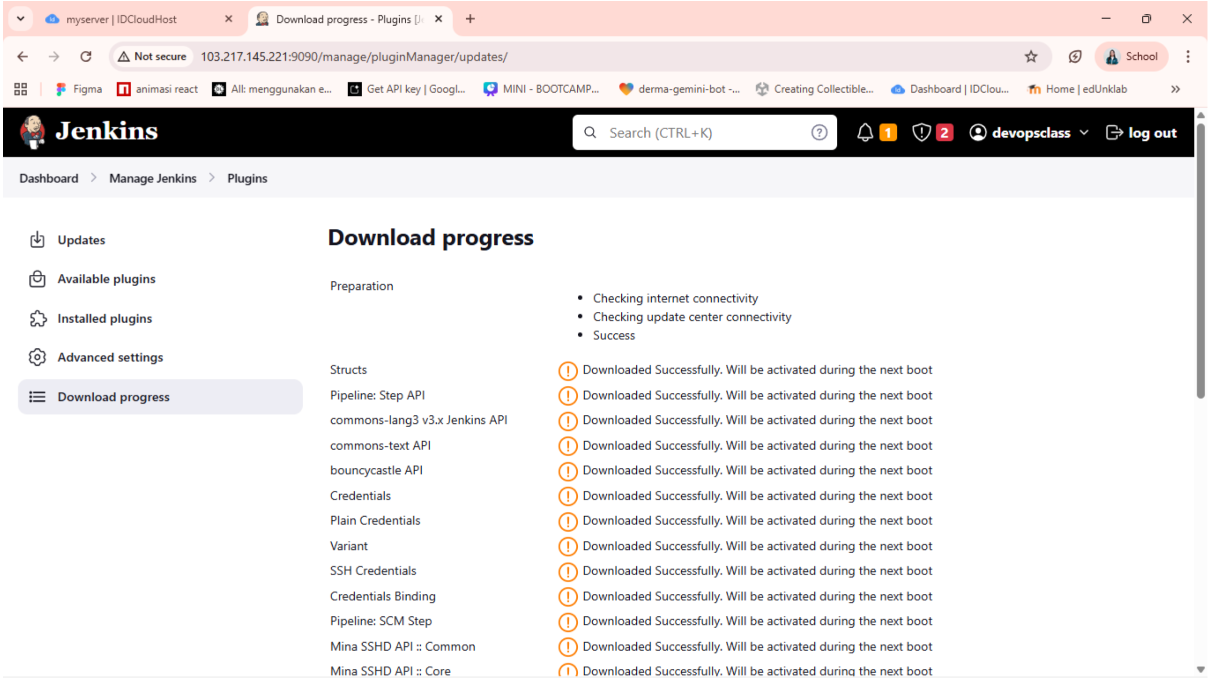


Pilih menu “Plugins”

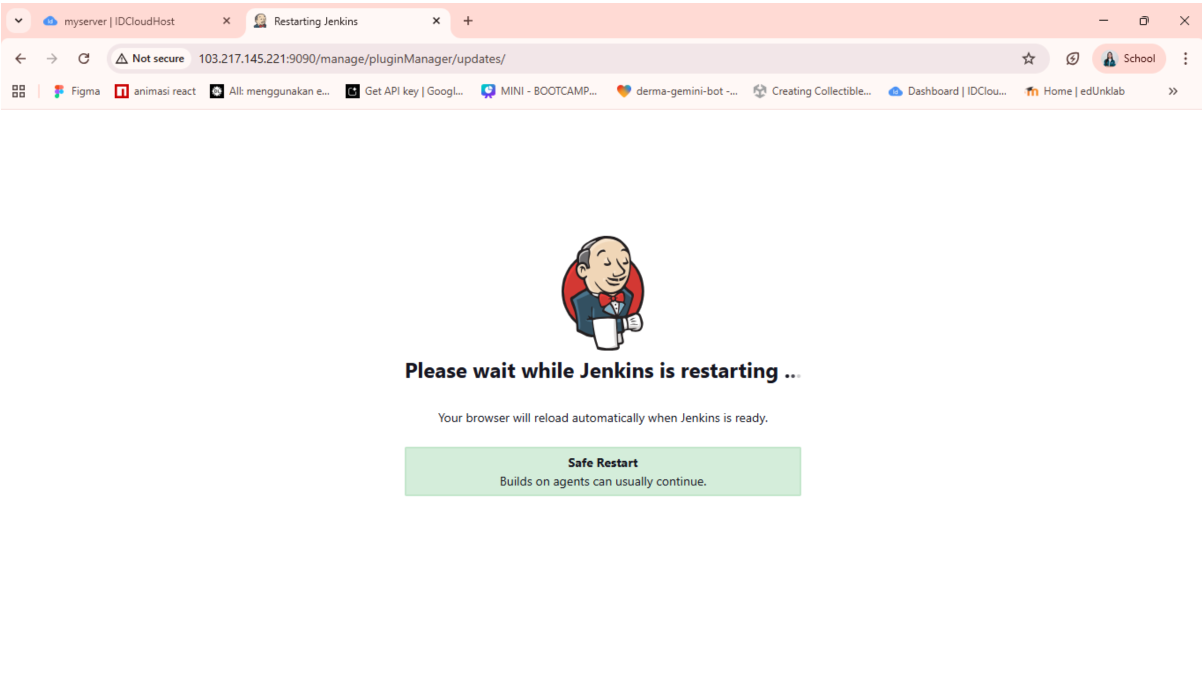


Pilih “Available plugins” pada sidebar dan cari “git”.
lalu centang pada row Git dan pilih “install after restart” pada tombol install.

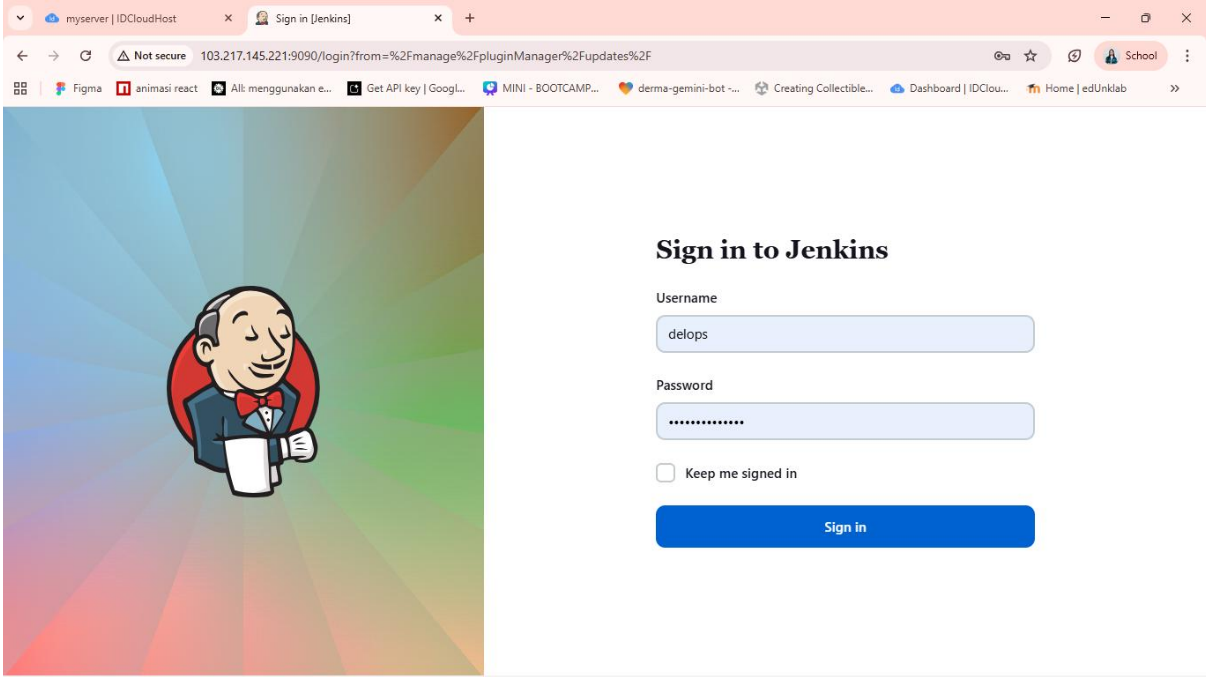




Setelah selesai download, scroll ke paling bawah dan centang pilihan "Restart Jenkins when installation is complete and no jobs are running".
lalu tampilan akan menjadi seperti ini:



Setelah selesai restart, login lagi ke dalam jenkins



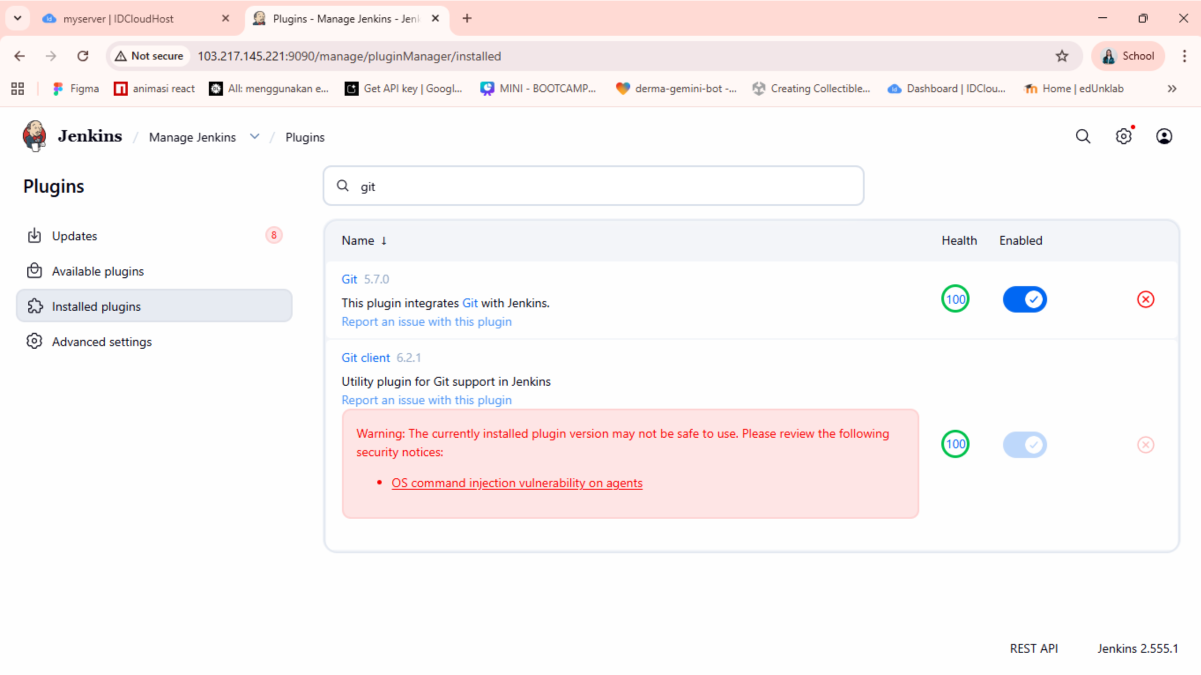
Note: kalau tampilan git di dalam "installed plugins" masih merah, berarti harus update jenkins war ke versi terbaru, lalu update juga java ke versi terbaru. Setelah itu jalankan service jenkins lagi.
Code Update jenkins: wget [https://get.jenkins.io/war-stable/latest/jenkins.war -O jenkins.war](https://get.jenkins.io/war-stable/latest/jenkins.war)

Code Update Java:

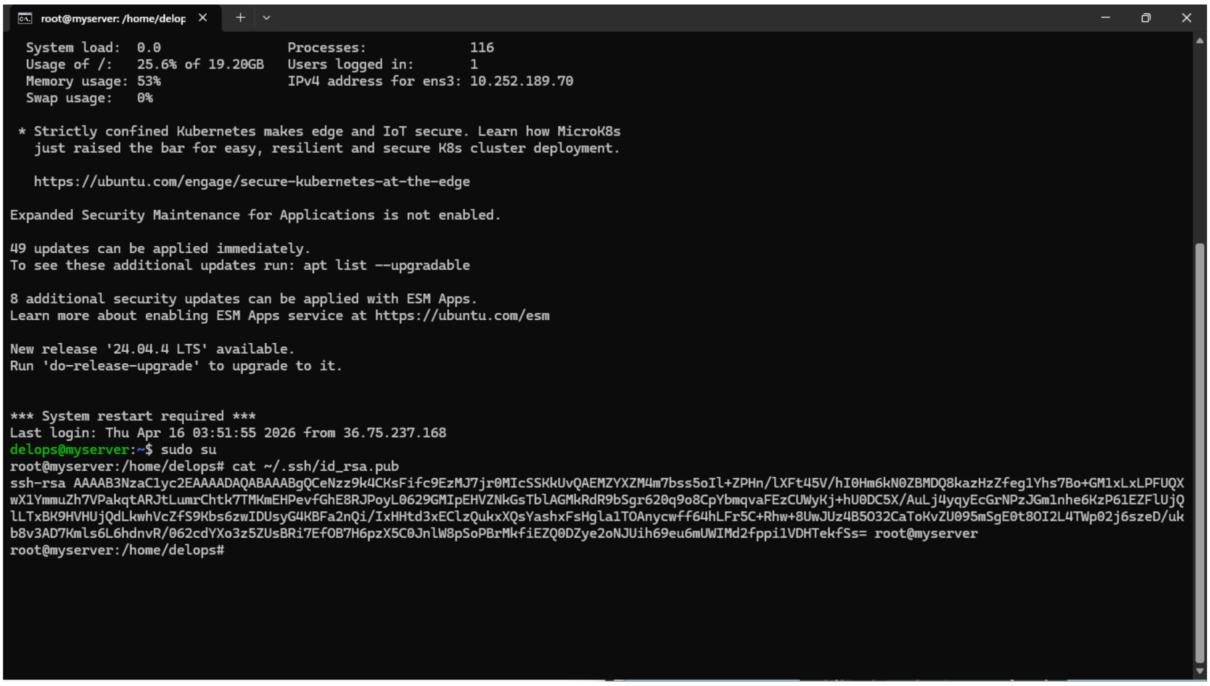
- sudo apt update
- sudo apt install fontconfig openjdk-21-jre -y
- sudo update-alternatives --config java → ini untuk ubah default java ke versi terbaru (pilih versi 21)
- java -version → cek versi Java yang sekarang

Jalankan kembali service jenkins: sudo java -jar jenkins.war --httpPort=9090 &

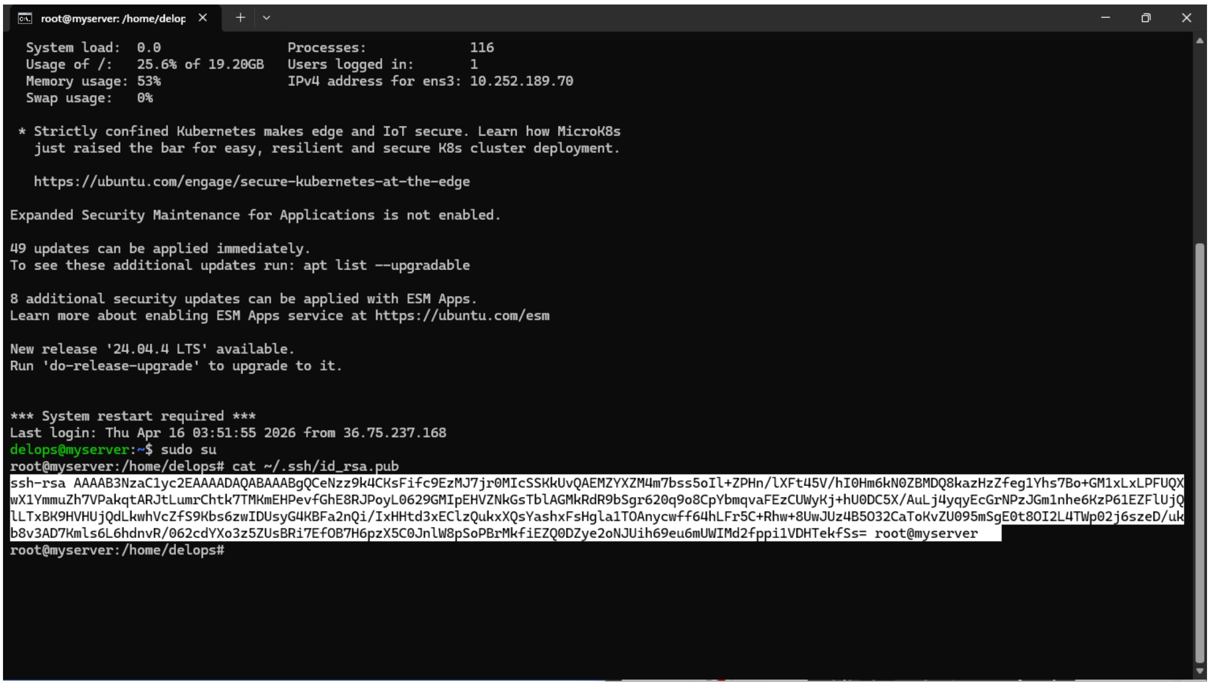
Maka tampilan dalam “installed plugins” akan jadi seperti ini:



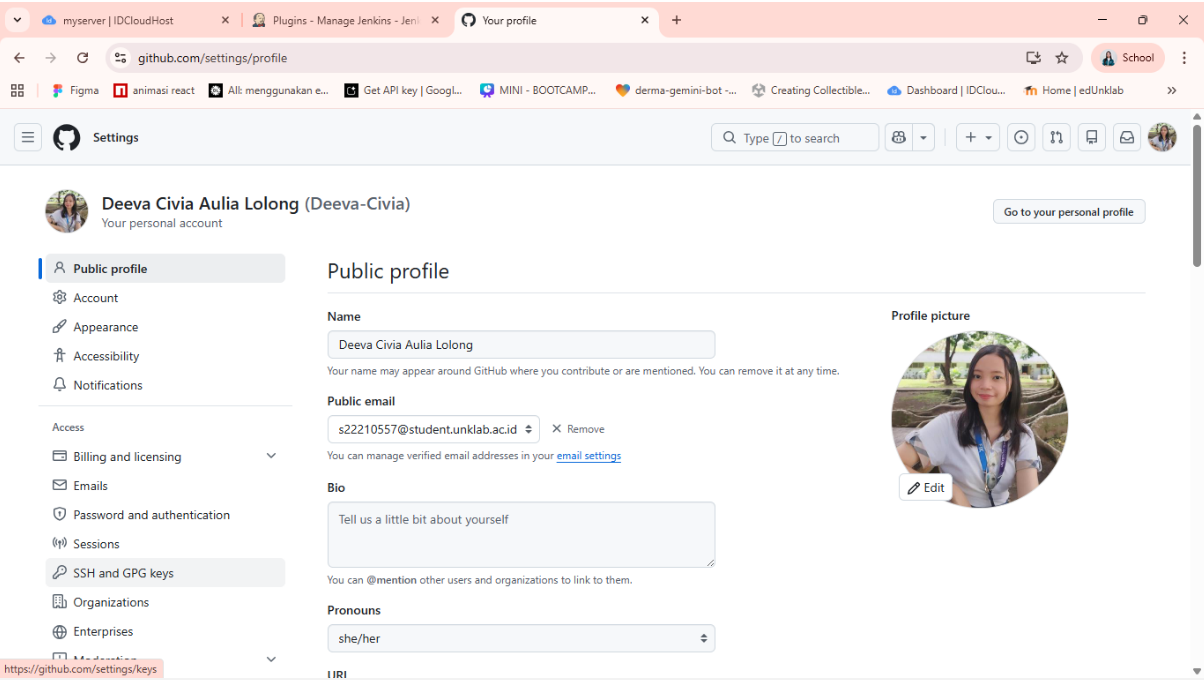
4. Cek Public Key



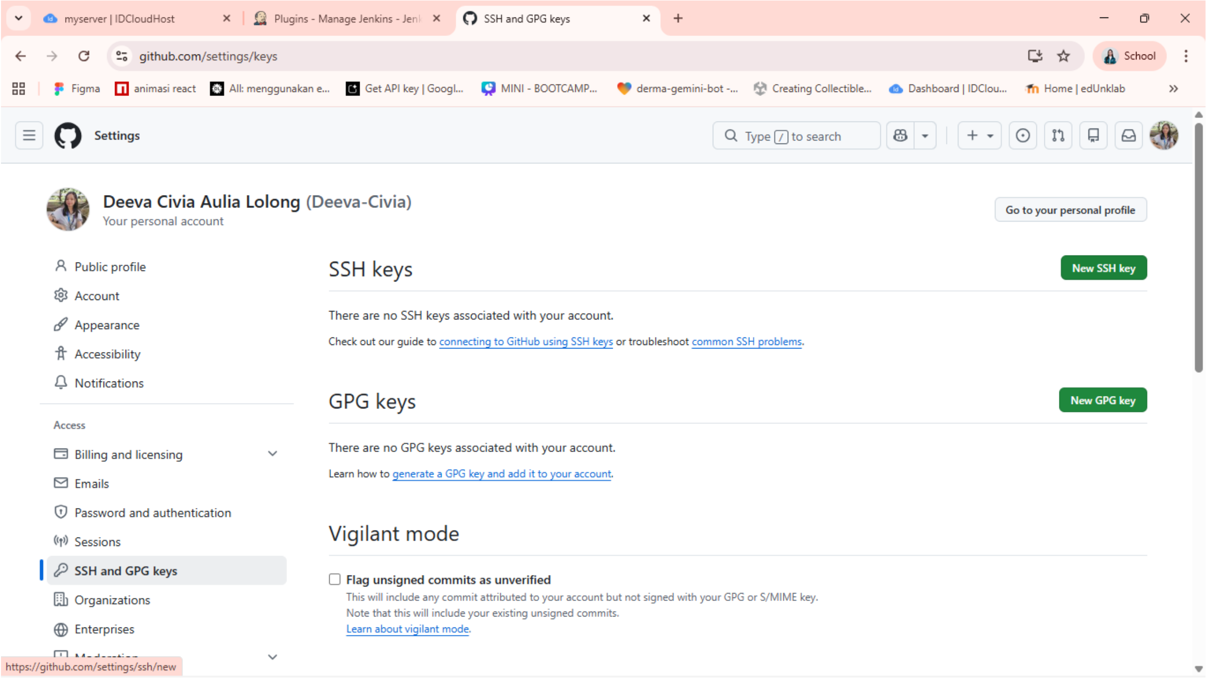
Copy public key ini, lalu paste ke dalam github



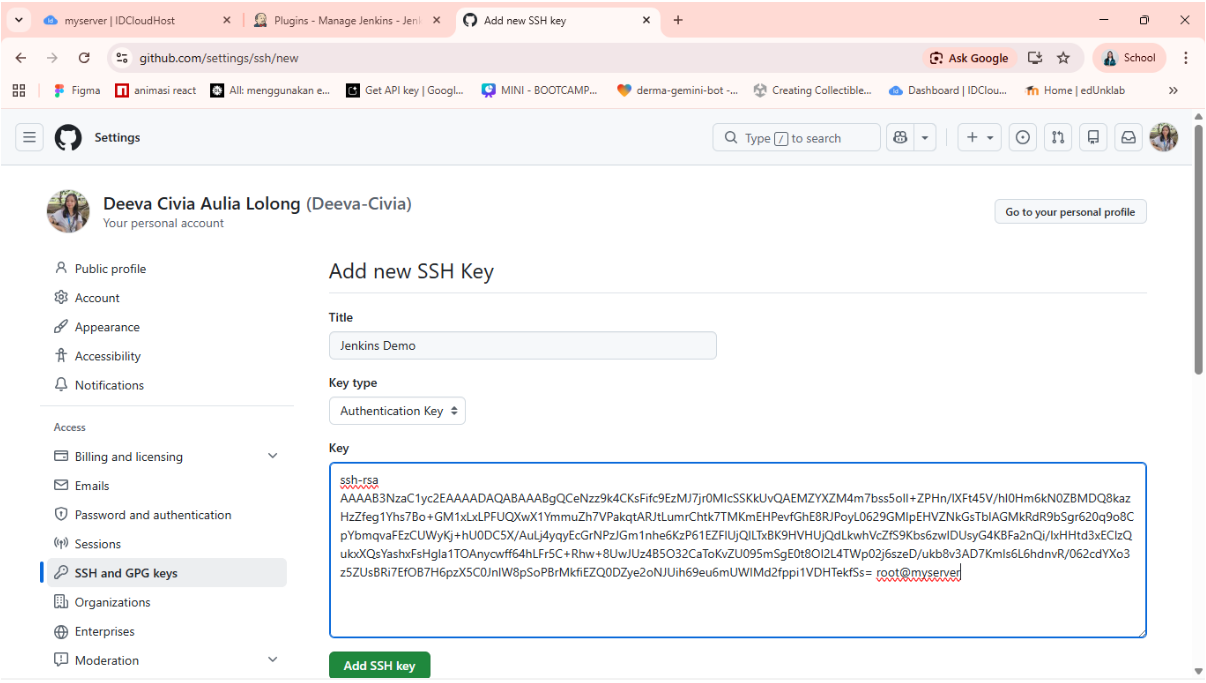
Buka Settings di profile Github



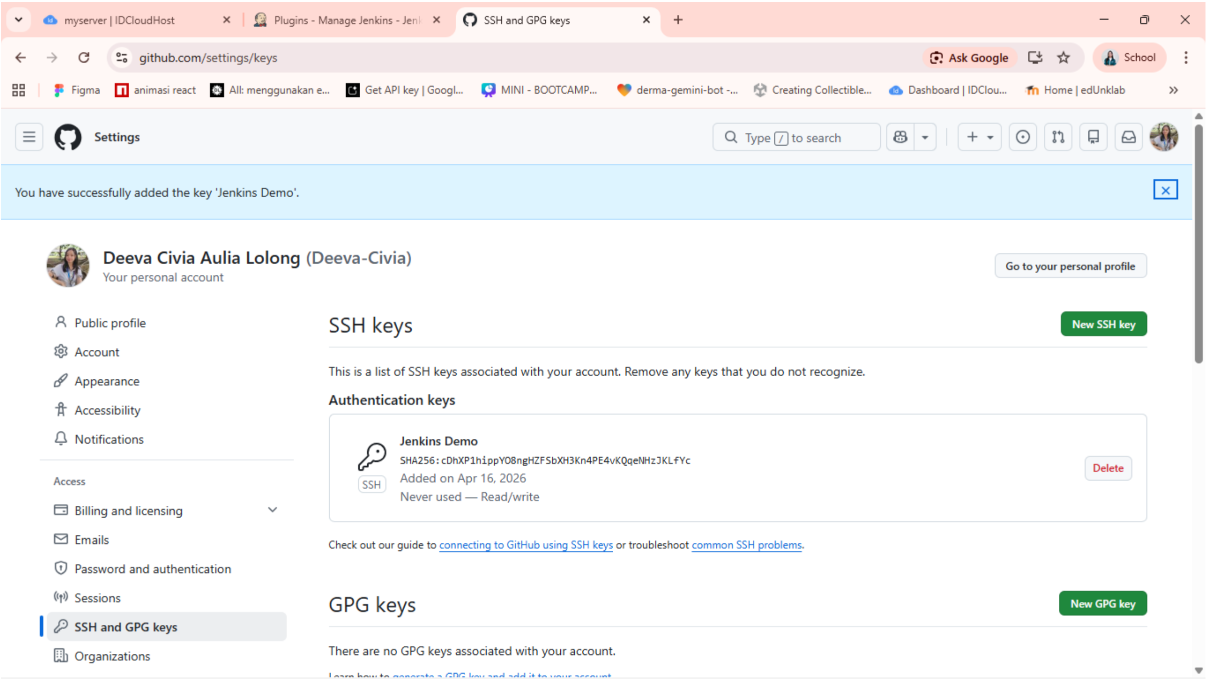
Pilih menu “SSH and GPG keys”. Lalu pilih “New SSH key”



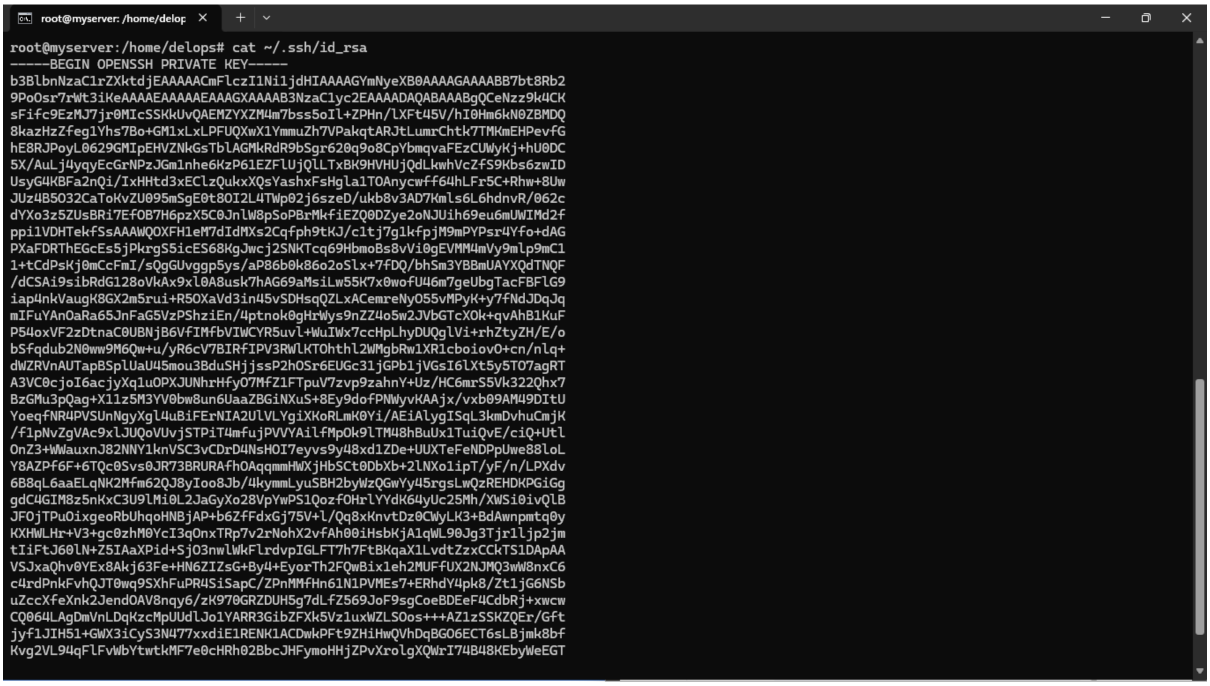
Masukkan public key yang telah di salin tadi ke dalam kolom “Key”, beri Title “Jenkins Demo”, lalu klik “Add SSH key”.



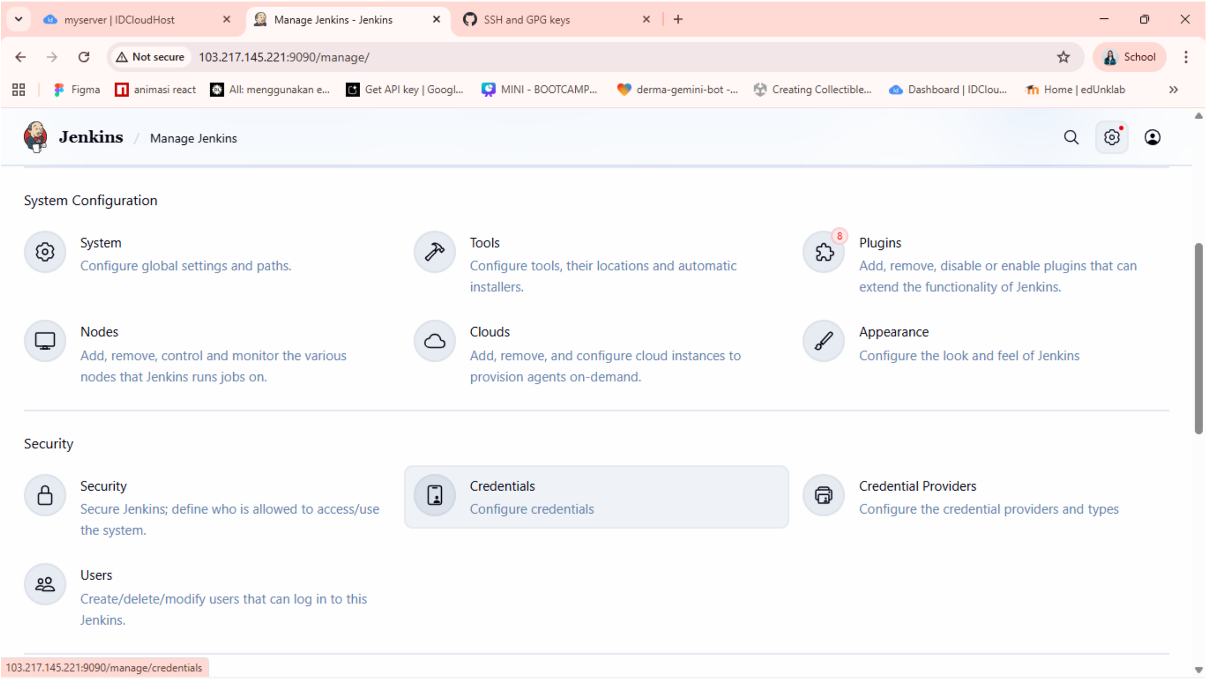
Kalau sudah seperti ini, berarti telah berhasil menyambungkan server dengan github melalui authentication keys.



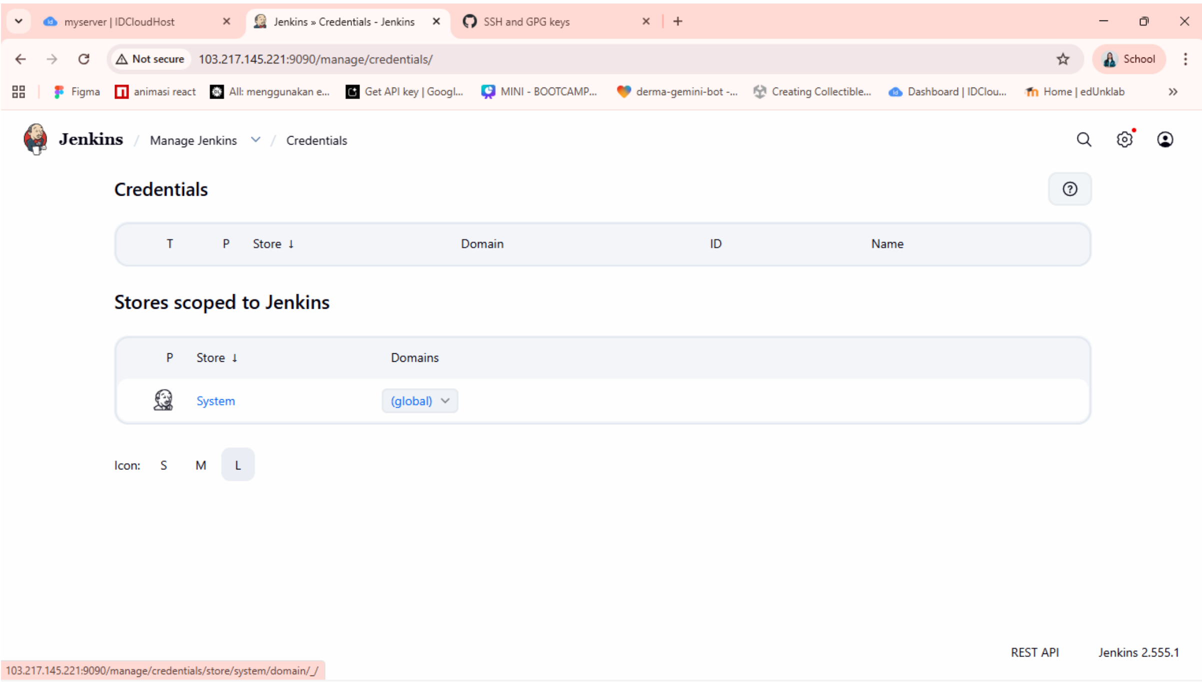
Sekarang kita ambil Private Key



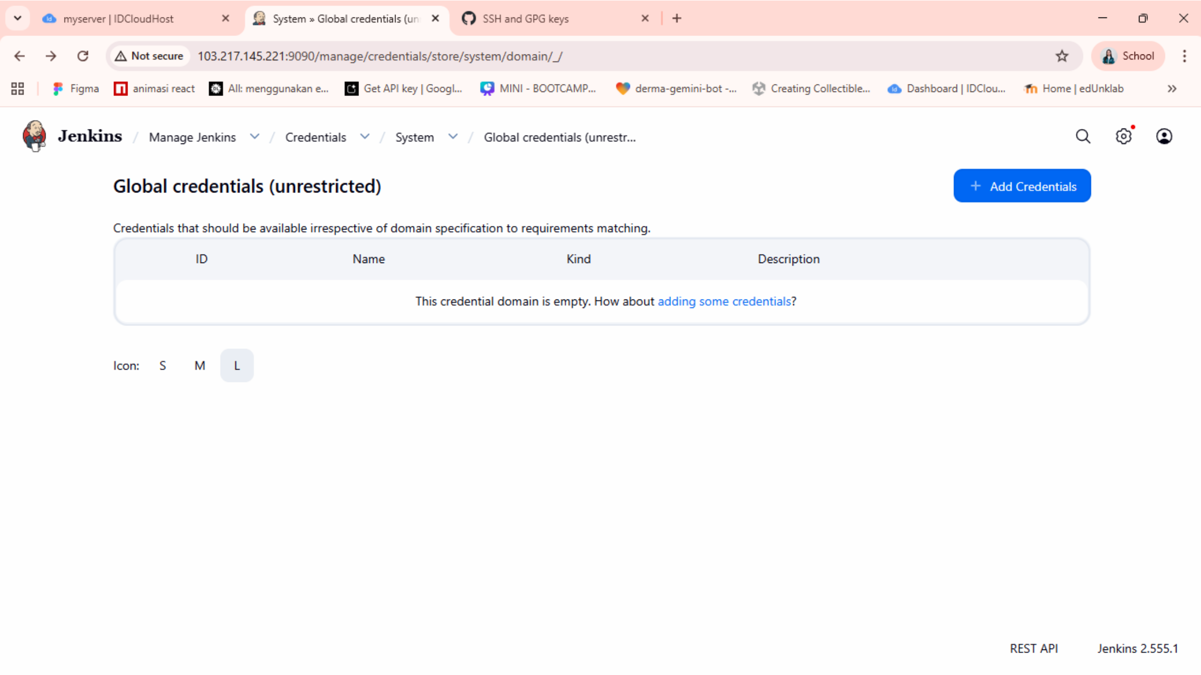
Buka Jenkins, dan buka menu “Menage Jenkins”, lalu pilih “Credentials”



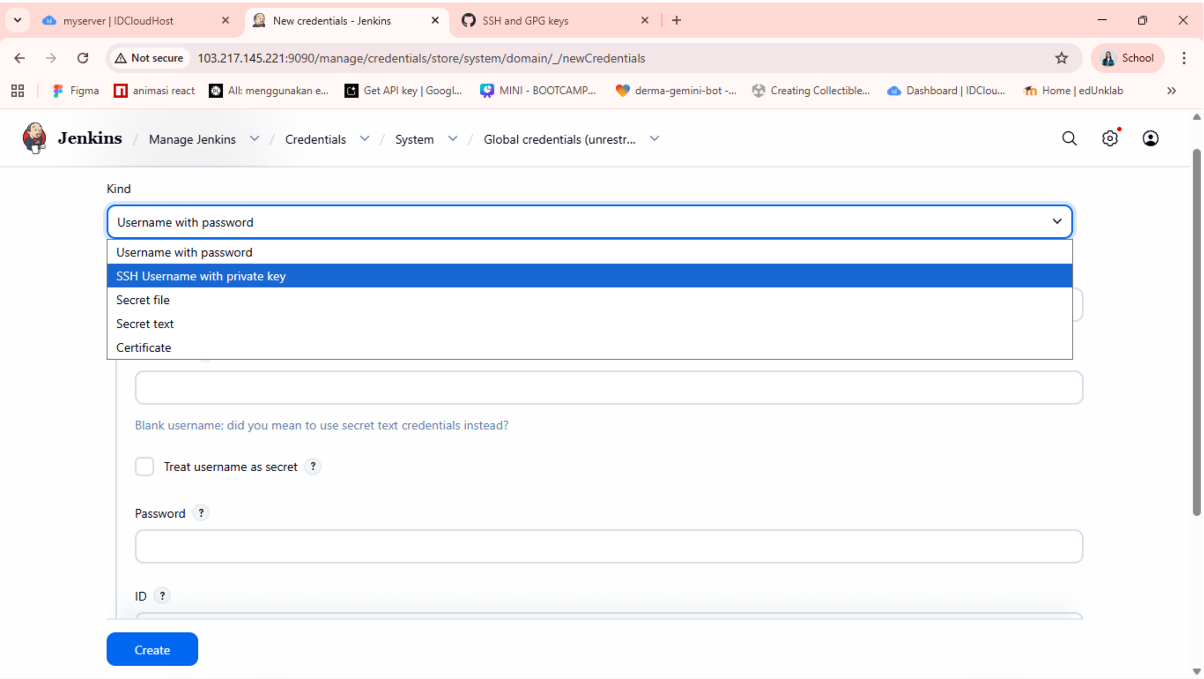
Pilih global, untuk membuat global credentials



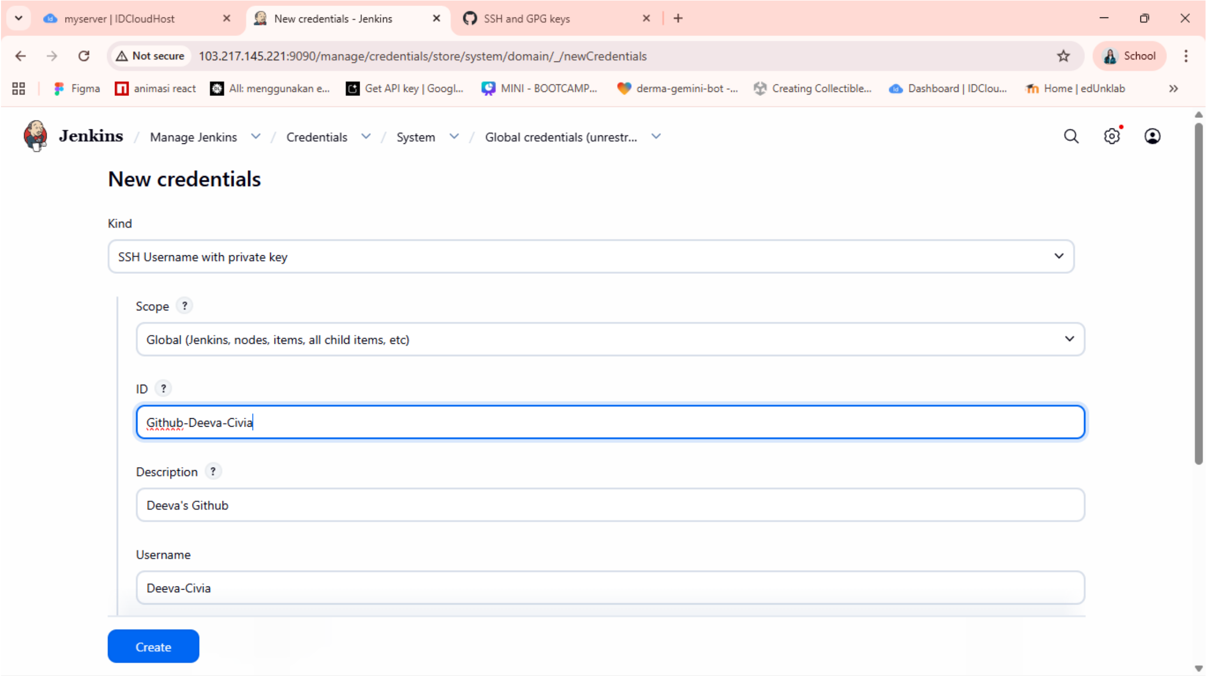
Klik “Add Credentials”



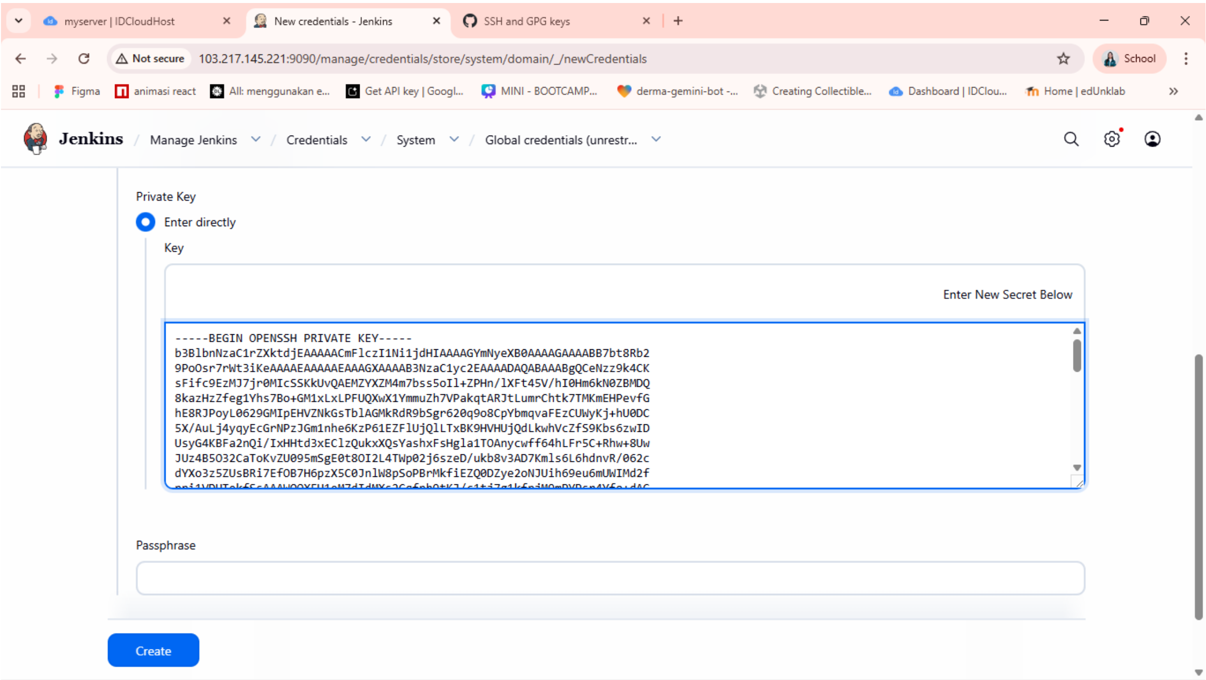
Dalam field “Kind”, pilih “SSH username with private key”



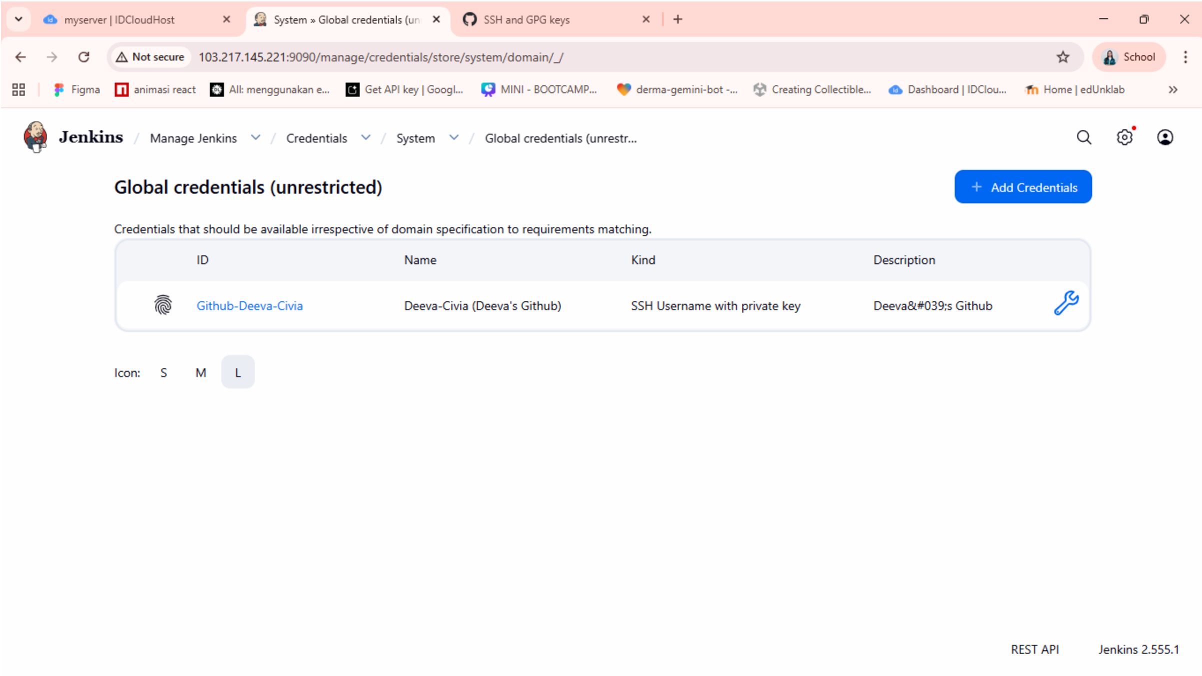
Masukkan username, untuk ID dan Description itu Optional.



Lalu pilih “Enter directly”, dan masukkan private key yang telah diambil dari server tadi. Dan klik Create

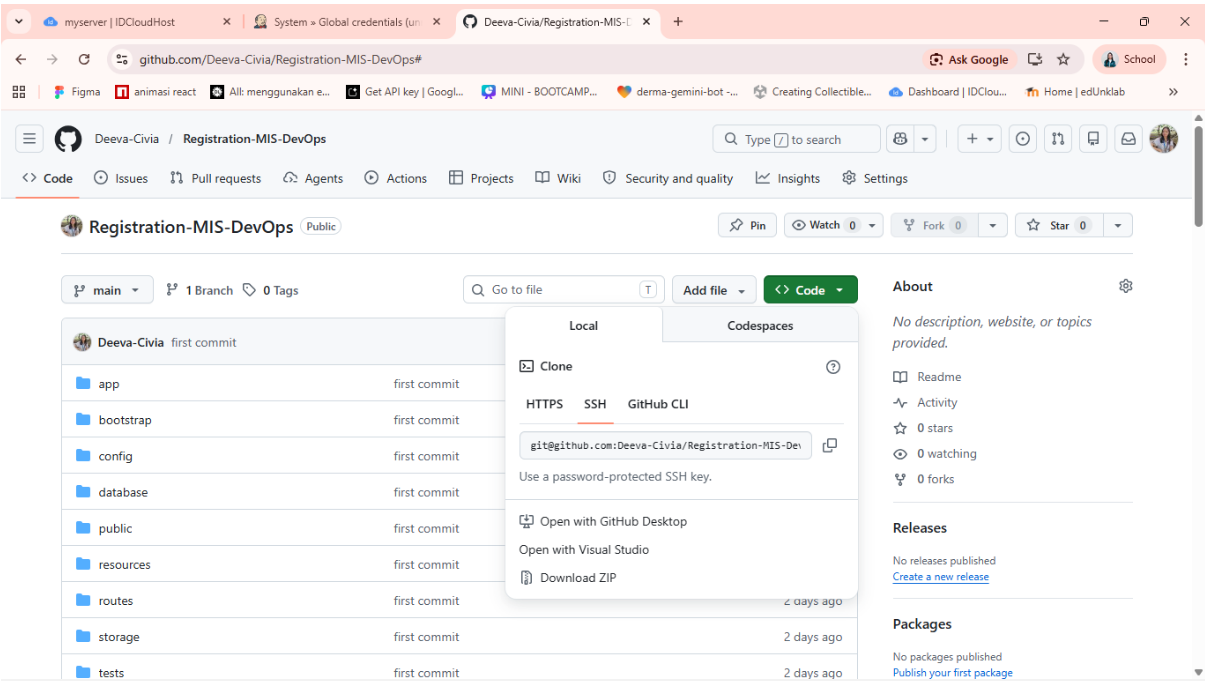


Ini tampilan credentials telah berhasil dibuat:

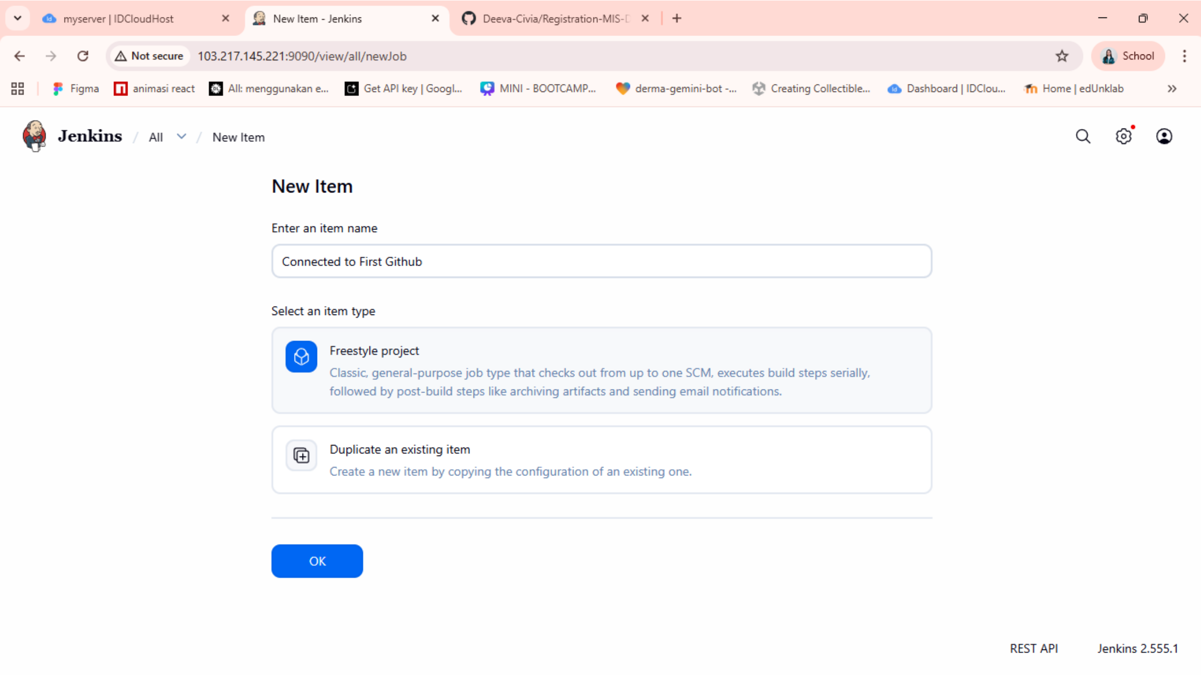


5. Integrasi Git dan Jenkins

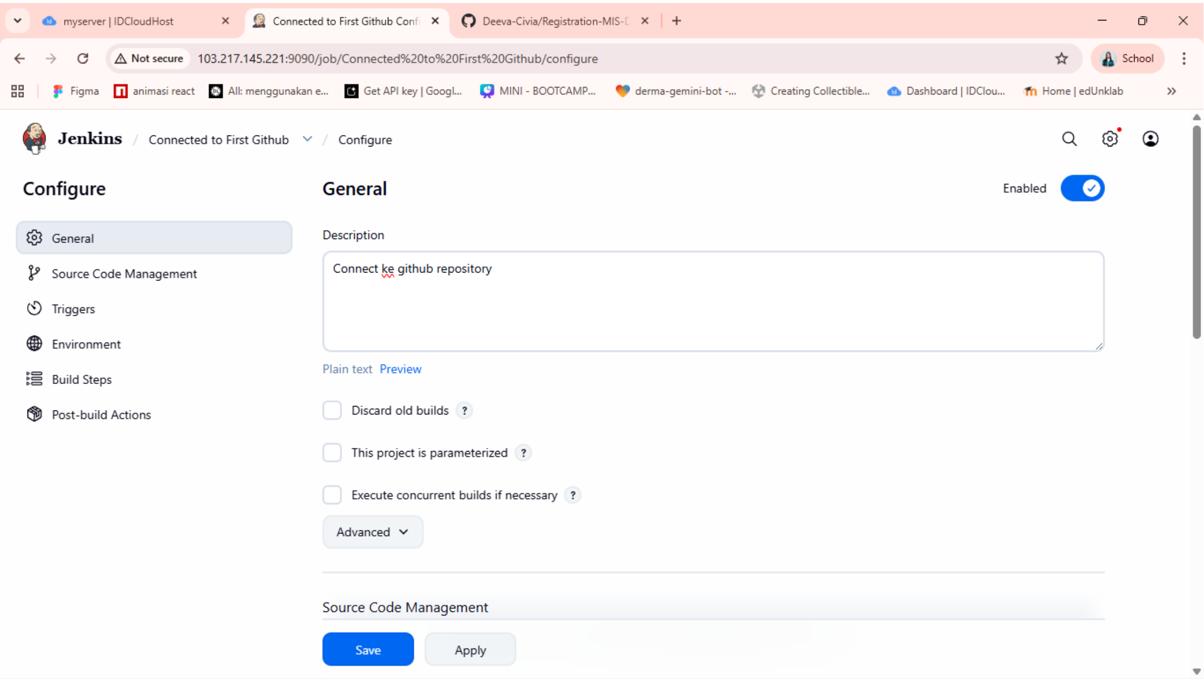
Buka project yang sudah dimasukkan ke github, lalu copy SSH link yang ada:



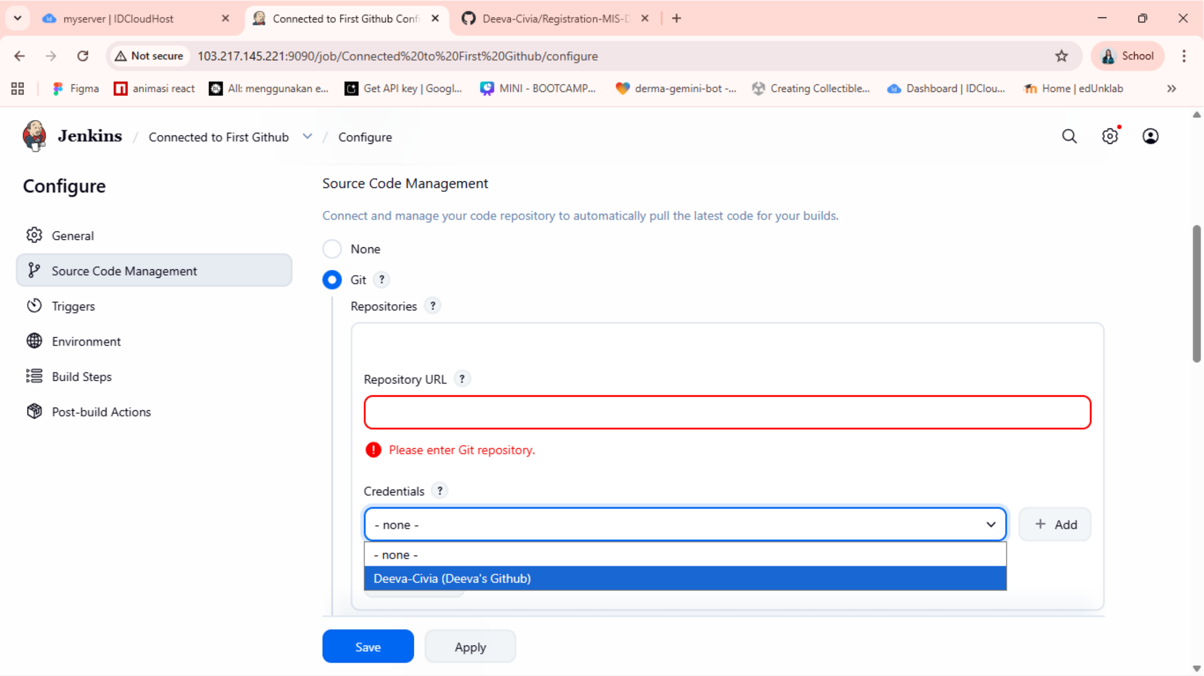
Buat job baru di jenkins, dengan nama “Connected to First Github”:



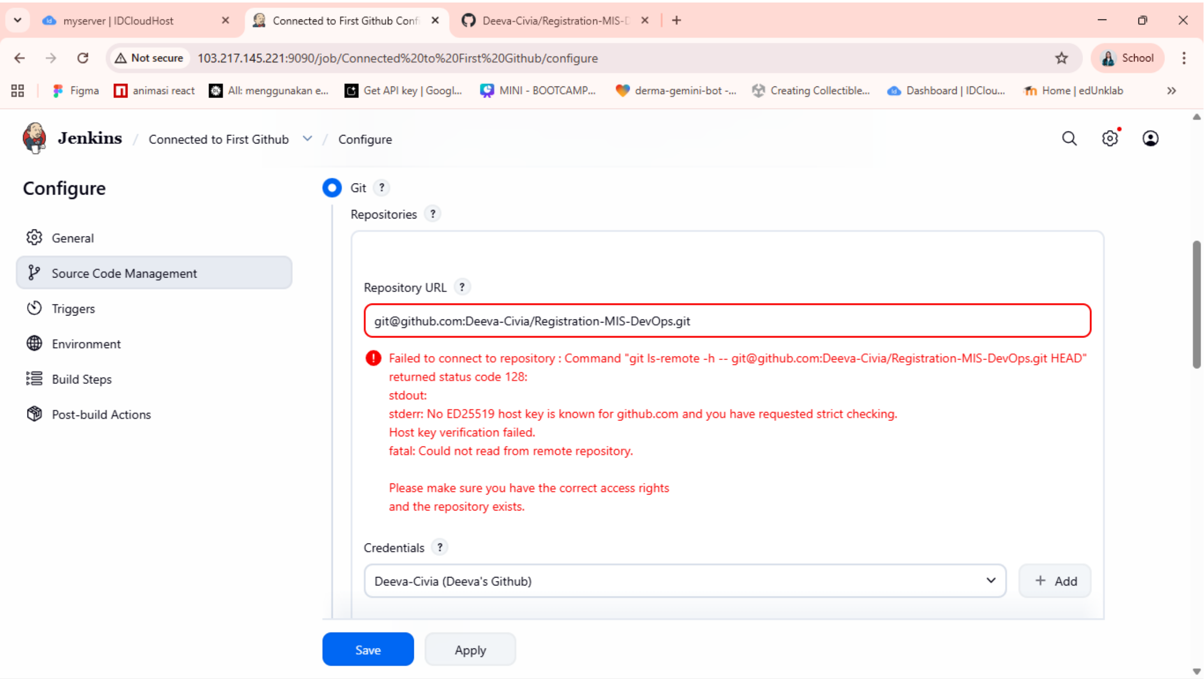
Masukkan Description:



Pada bagian “Source Code Management” pilih credentials yang telah dibuat tadi:



Lalu “Repository URL” masukkan SSH link yang telah di copy dari project di github.



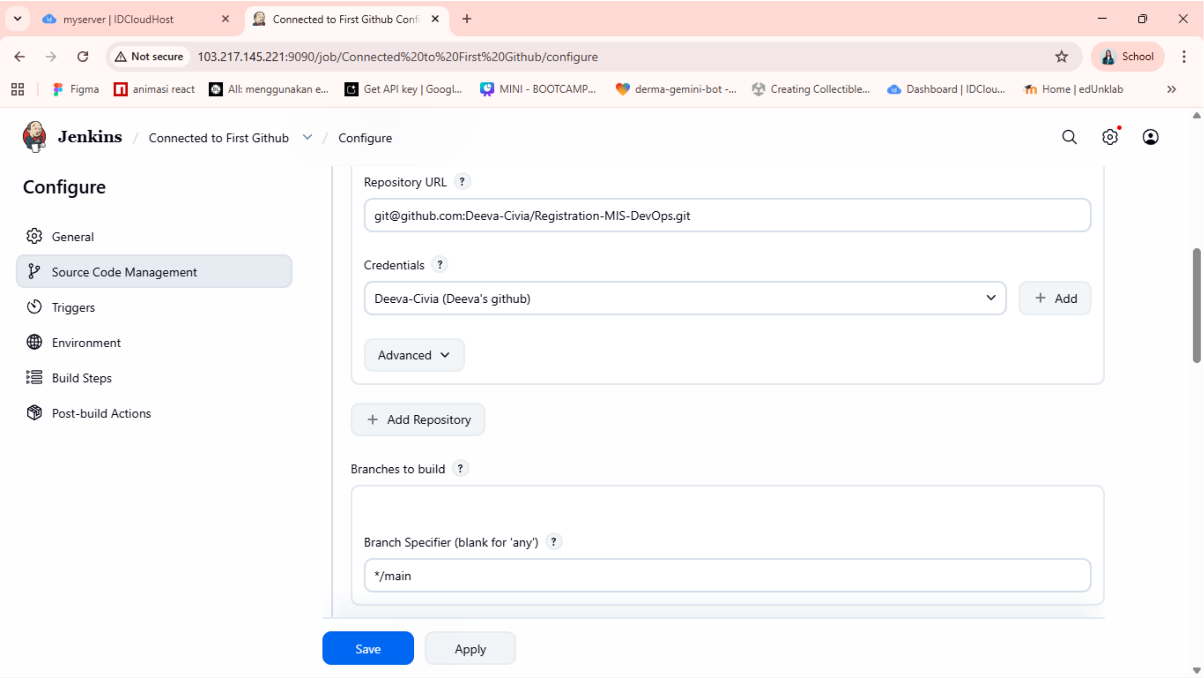
Kalau muncul failed seperti ini, berarti server Jenkins belum mengenali atau belum percaya pada identitas server GitHub (github.com). Hal ini karena secara bawaan, protokol SSH menerapkan pengecekan ketat. Ketika Jenkins mencoba menghubungi GitHub untuk pertama kalinya, GitHub akan mengirimkan *host key*. Karena kunci ini belum ada di dalam daftar known_hosts di server, dan Jenkins berjalan di latar belakang, koneksi tersebut otomatis ditolak oleh sistem.

Maka kita harus menjalankan perintah ini di dalam server:

- mkdir -p ~/.ssh → Membuat folder .ssh di server. Folder ini untuk menyimpan semua konfigurasi dan kunci keamanan.
- ssh-keyscan github.com >> ~/.ssh/known_hosts → Mendaftarkan GitHub sebagai server yang aman dan dipercaya. Perintah ini secara otomatis memindai identitas keamanan (kunci publik) dari github.com, lalu menyalinnya ke dalam file daftar kontak tepercaya (known_hosts) supaya koneksi selanjutnya tidak ditolak

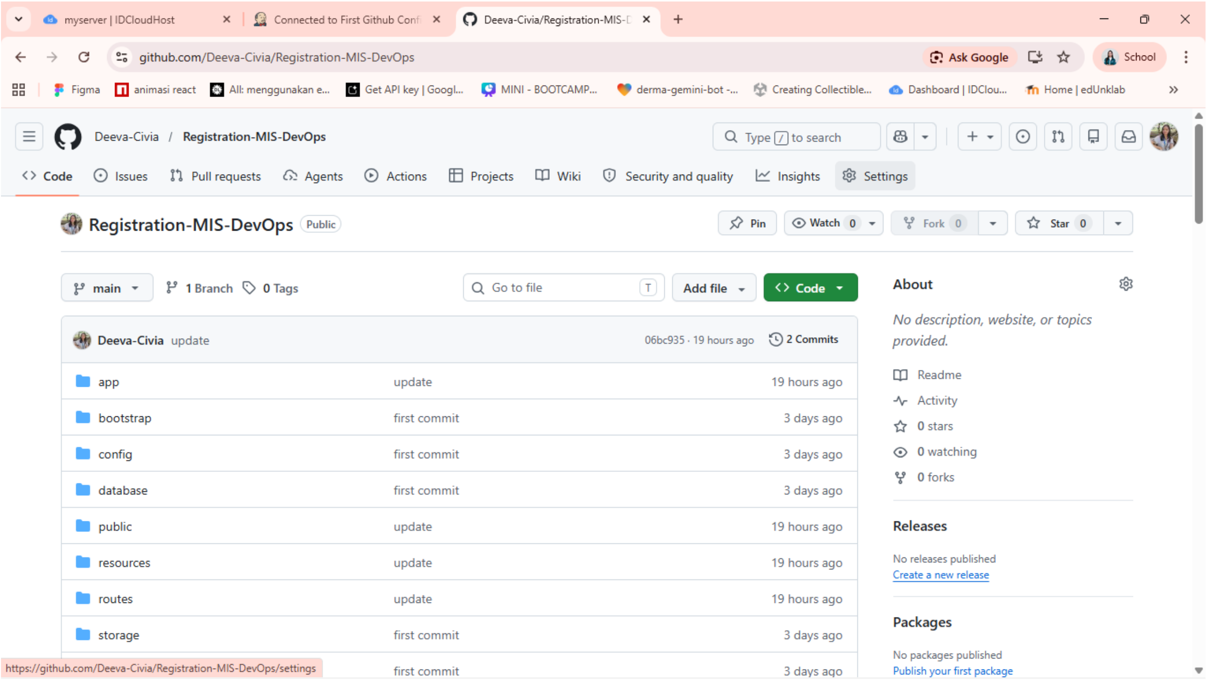
```
root@myserver:/home/delops# mkdir -p ~/.ssh
root@myserver:/home/delops# ssh-keyscan github.com >> ~/.ssh/known_hosts
# github.com:22 SSH-2.0-3992d52
# github.com:22 SSH-2.0-3992d52
# github.com:22 SSH-2.0-3992d52
# github.com:22 SSH-2.0-3992d52
# github.com:22 SSH-2.0-3992d52
root@myserver:/home/delops#
```

Masukkan nama branch yang ada di github yaitu “main”, lalu klik save.

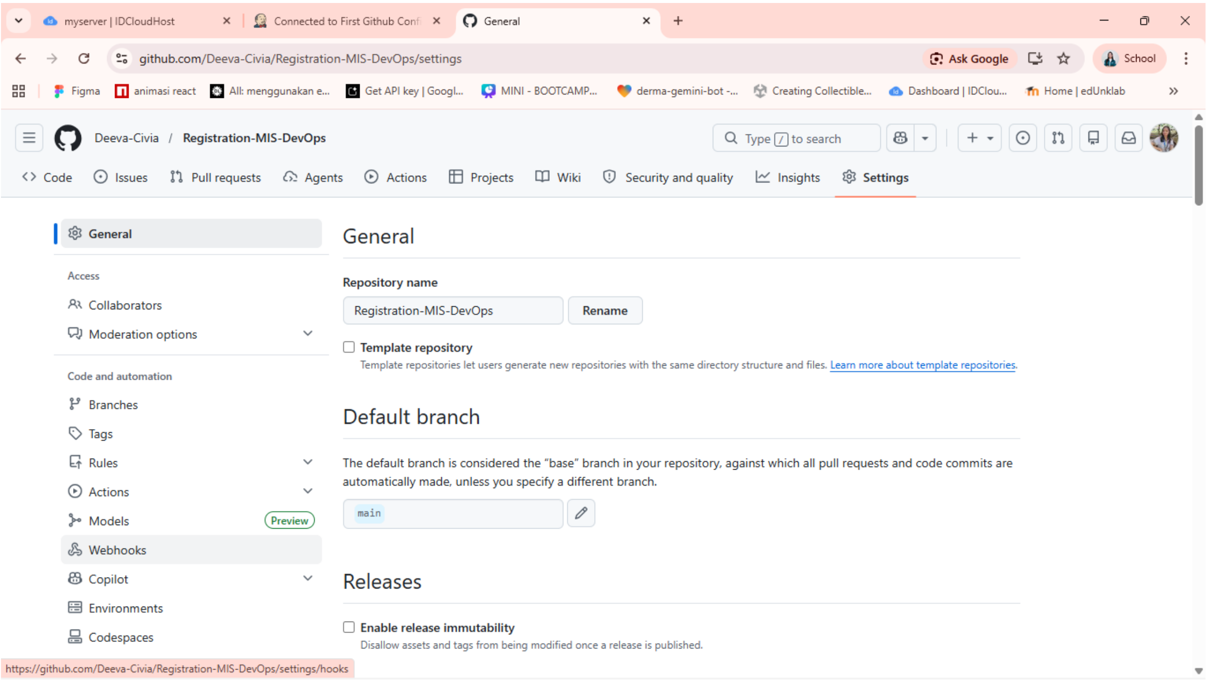


Selanjutnya kita akan membuat Webhook.
Webhook disini berperan sebagai alarm yang akan melakukan trigger atau memberi tahu Jenkins jika kita baru saja melakukan push dari local ke github. Hal ini agar jenkins tidak perlu melakukan pengecekan tiap berapa menit, melainkan hanya akan mengambil perubahan jika memang ada push terbaru.

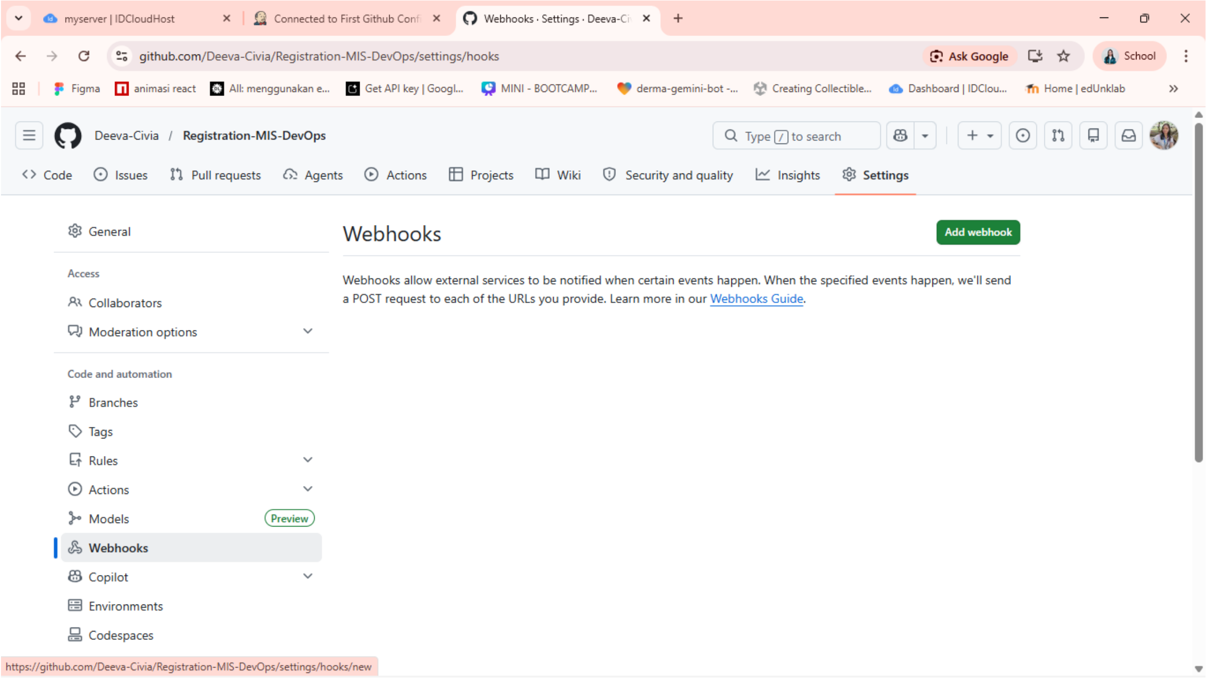
Buka Repository Github, dan pilih menu “Settings” yang ada di dalam repository



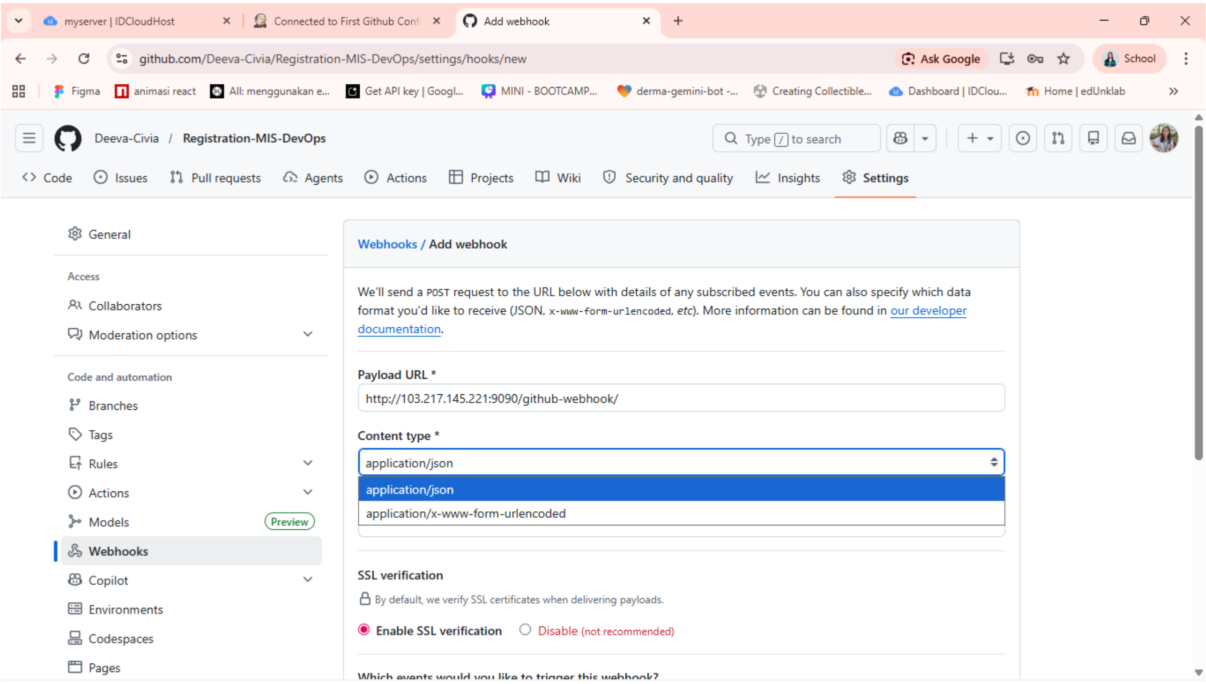
Pada menu settings ini, pilih “Webhooks” yang ada pada side bar:



Dalam menu Webhooks, klik “Add webhook”

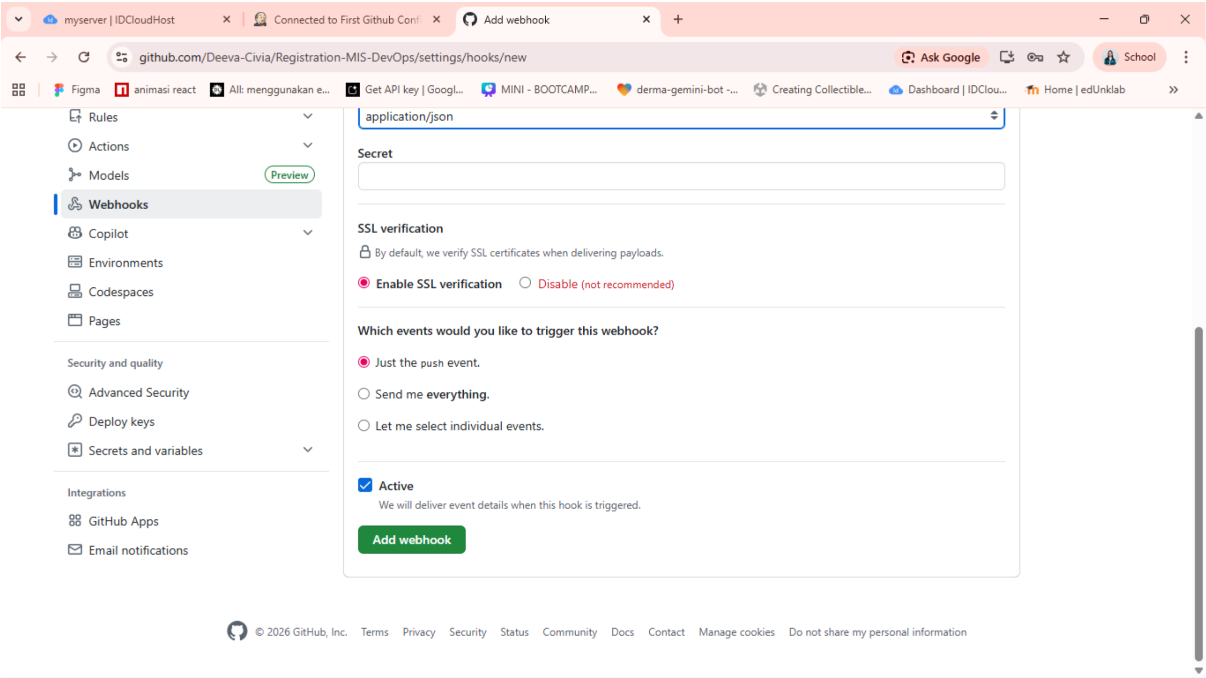


Untuk bagian ini, pada Payload URL diisi dengan IP server dengan port Jenkins yaitu
http://<IP_SERVER >:9090/github-webhook/
dan untuk bagian Content Type, pilih “application/json”.

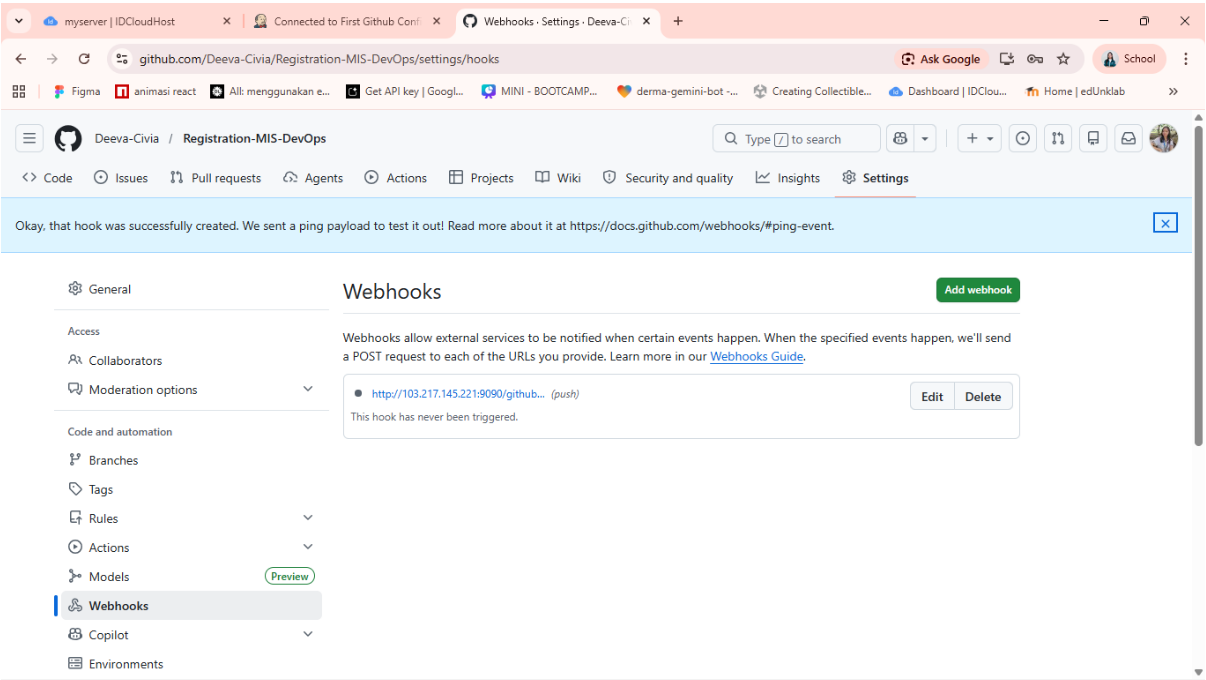


Dan pada bagian “Which events would you like to trigger this webhook?”, pilih “Just the push event”

Lalu untuk pengaturan lainnya dibiarkan default seperti itu, dan klik “Add webhook”

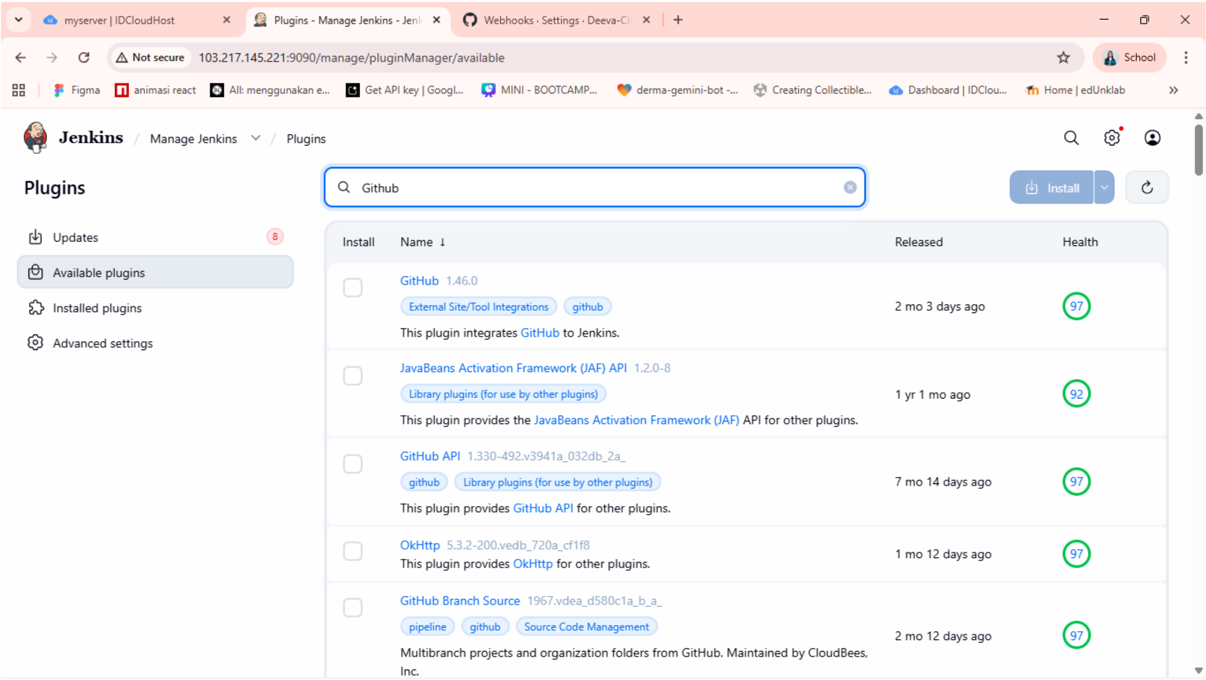


Tampilan setelah Webhook berhasil dibuat:

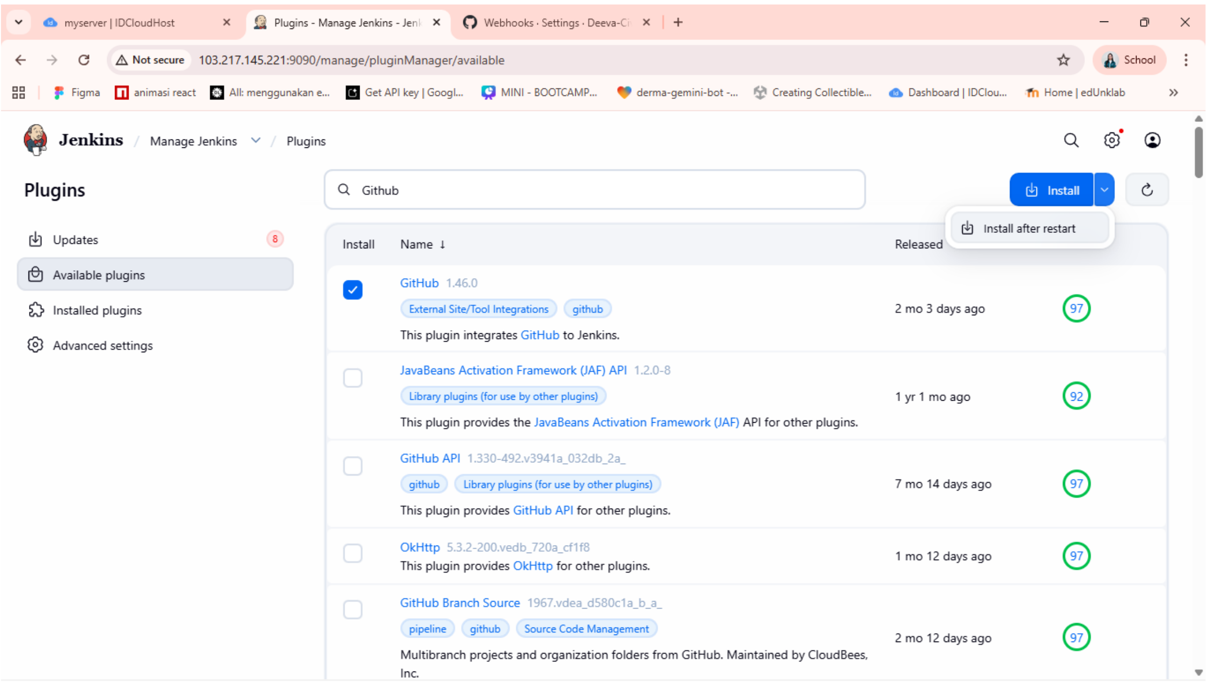


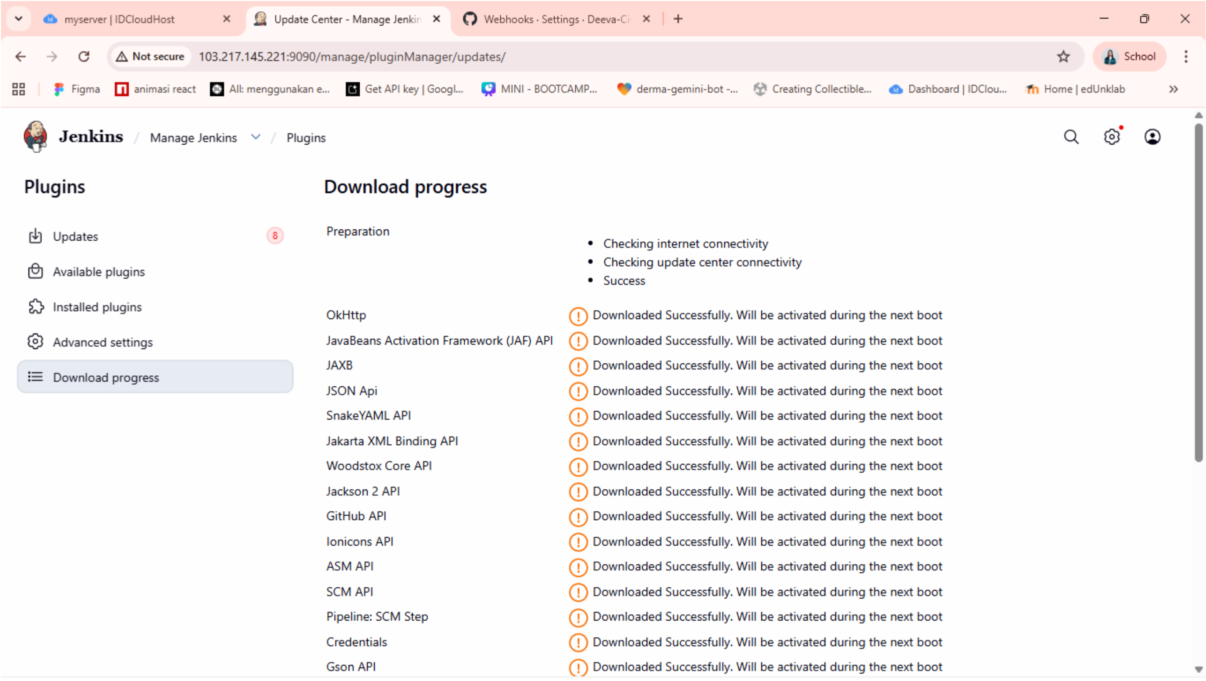
Sekarang kita beri tahu Jenkins untuk selalu mendengarkan alarm dari Github, dengan cara mengatur trigger pada job yang telah kita buat tadi.

Namun sebelum itu, kita harus melakukan install plugin khusus untuk Github.
Caranya, kembali ke menu Pluggins, dan masuk ke "Available pluggins" lalu cari "Github" pada kolom pencarian.

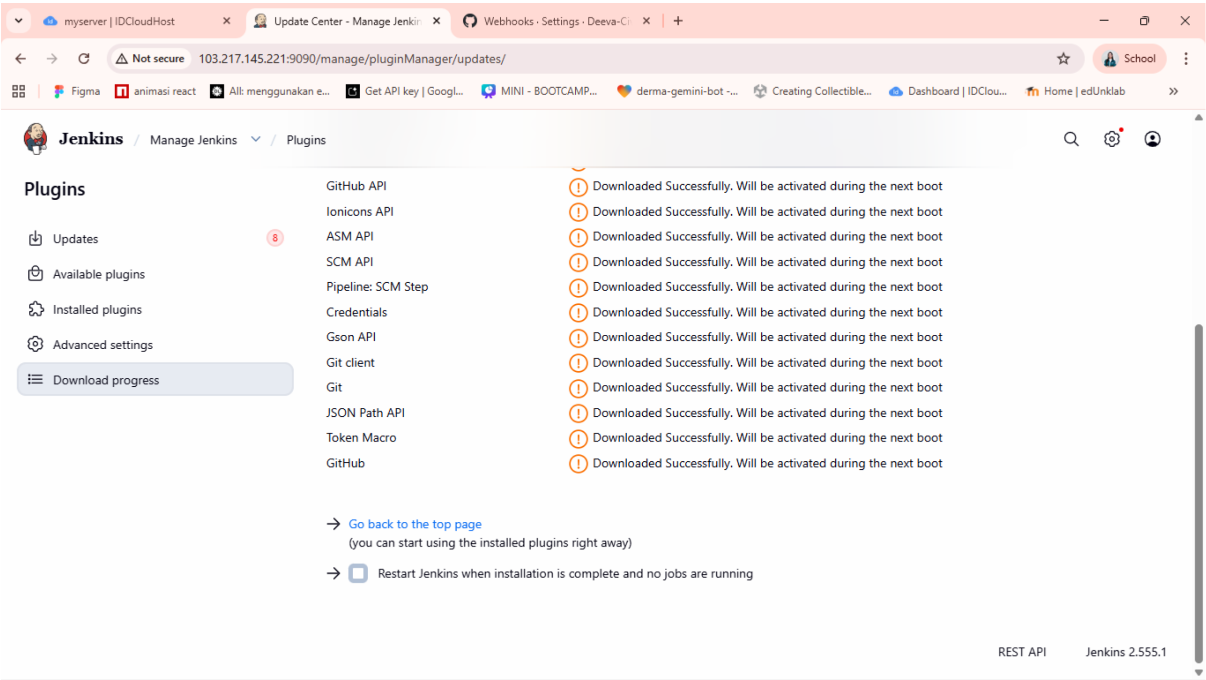


Centang lalu klik "Install after restart"





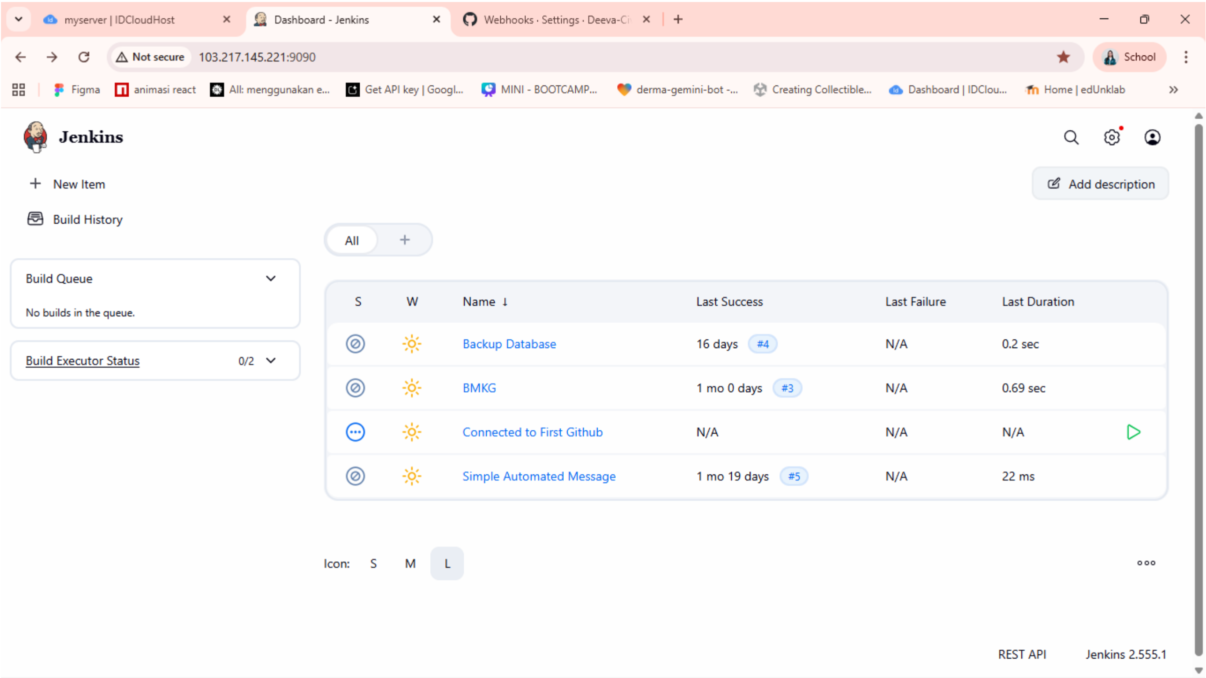
Centang bagian “Restart Jenkins when installation is complete and no jobs are running”



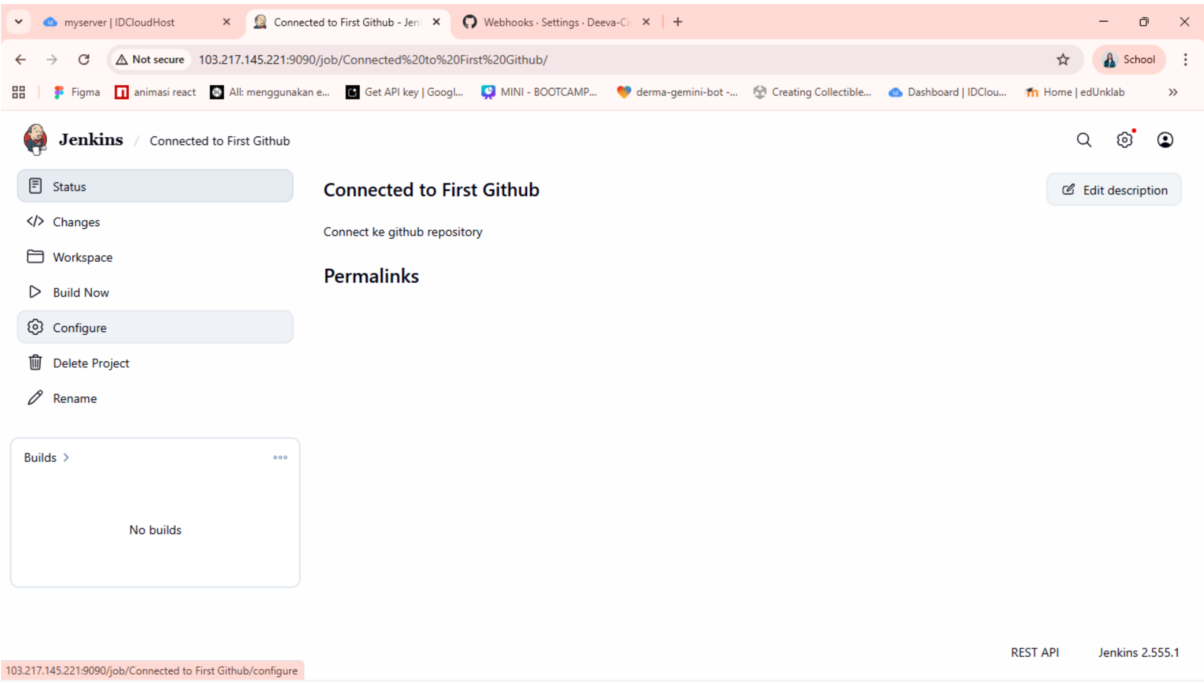
Setelah itu, Jenkins akan melakukan restart.

Selesai restart, kita login kembali, dan masuk ke menu Job yang telah kita buat tadi.

“Connected to First Github”

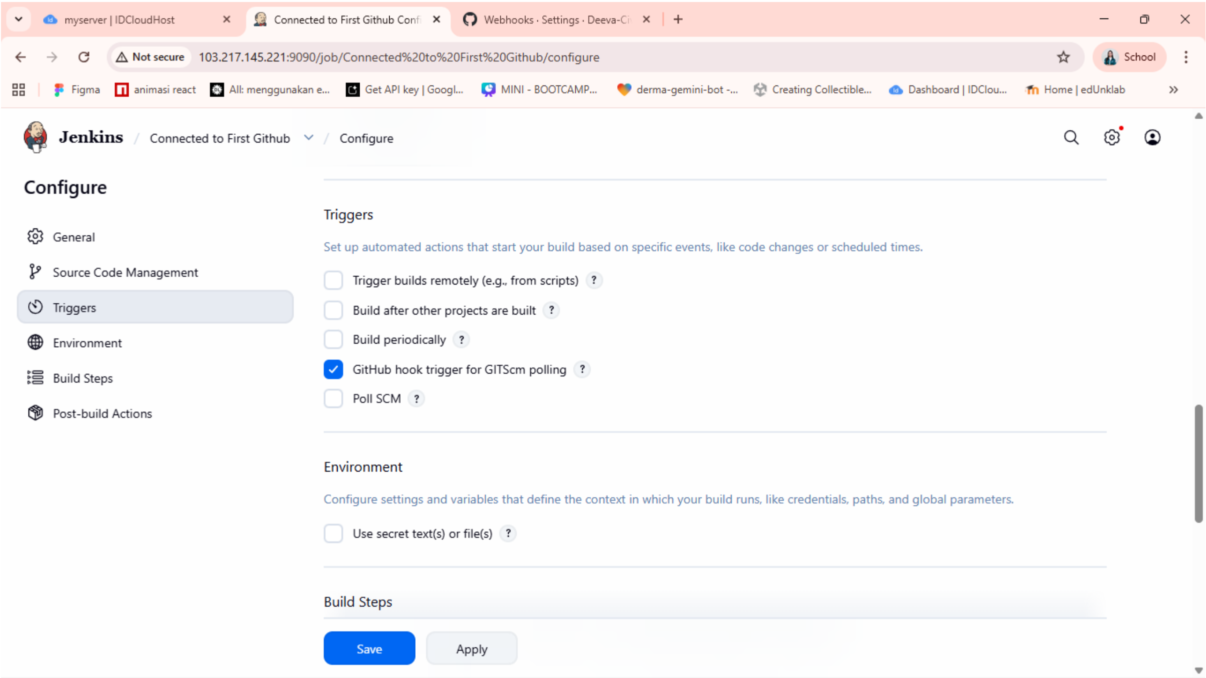


Klik “Configure”



Dalam menu Configure, pilih menu “Triggers” yang ada pada side bar

Centang bagian “GitHub hook trigger for GITScm polling”, lalu klik save



Selanjutnya saat Jenkins mendapat code terbaru dari GitHub, code itu masuk ke dalam Workspace internal Jenkins. Kita harus menyuruh Jenkins memindahkan code tersebut ke directory project website kita yaitu /var/www/html/registration-mis

Caranya, masuk lagi ke menu Configure dan pilih menu "Build Step" pada side bar, dan pilih "Execute shell".
Dalam field Execute shell, masukkan code dibawah ini:

rsync -avz --exclude '.git' --exclude '.env' ./ /var/www/html/registration-mis/ → untuk sinkronisasi file dari internal Jenkins (workspace) ke direktori web server tujuan.

Detail Parameter:

- rsync: Perintah transfer file yang lebih cepat karena hanya mengirimkan bagian file yang berubah
- -a (archive): Menjaga struktur direktori, hak akses (permissions), dan symlink file tetap sama seperti aslinya
- -v (verbose): Menampilkan proses transfer file di log Jenkins
- -z (compress): Compress data selama proses transfer supaya lebih ringan dan cepat
- --exclude '.git': Mencegah folder riwayat GitHub ikut tersalin ke web server publik
- --exclude '.env': Mencegah file env ikut tertimpa.
- ./: Direktori sumber yaitu workspace Jenkins saat ini
- /var/www/html/registration-mis/: Direktori tujuan tempat project website

cd /var/www/html/registration-mis/ → untuk masuk ke dalam folder project Laravel yang ada di server.

composer install --optimize-autoloader --no-dev → Mengelola dan menginstal library PHP yang dibutuhkan Laravel.

php artisan migrate –force → Menjalankan file migrasi PHP yang baru ditambahkan ke repositori untuk memperbarui skema database di server. Menggunakan Force agar langsung mengeksekusi migrasi.

chown -R www-data:www-data /var/www/html/registration-mis → Mengubah kepemilikan folder dan seluruh isinya kepada user www-data dan group www-data.

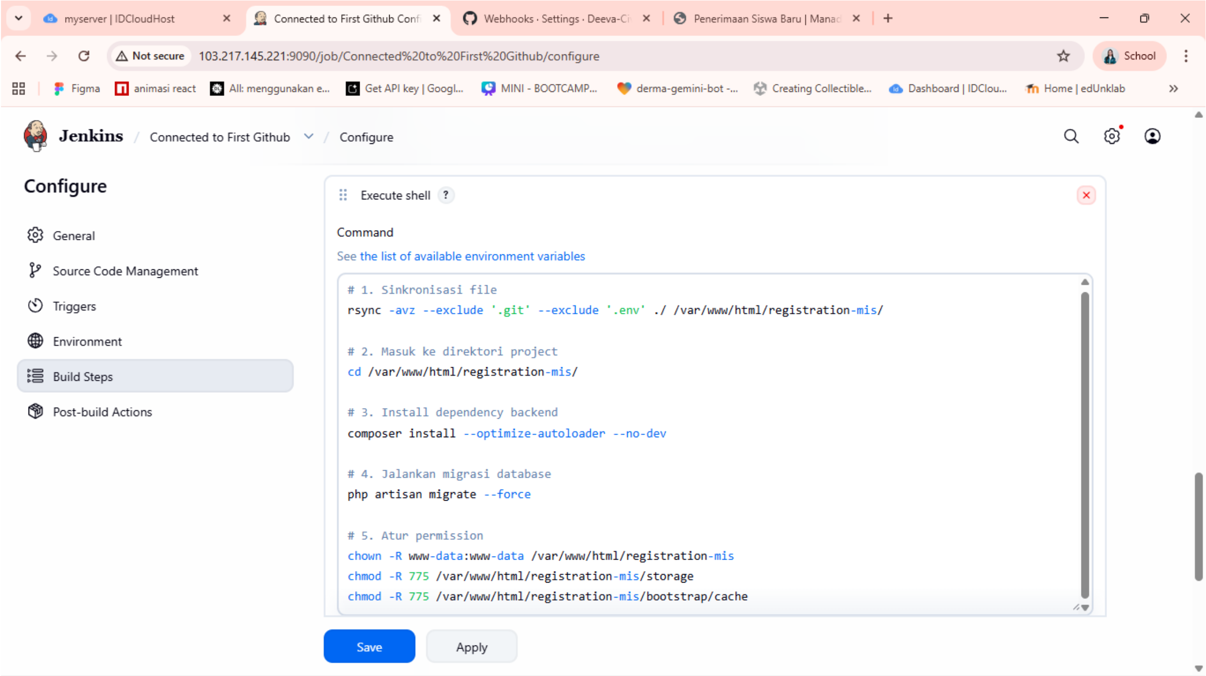
www-data adalah user standar Linux untuk web server, sehingga web server memiliki hak untuk membaca project tersebut.

chmod -R 775 /var/www/html/registration-mis/storage → Mengubah hak akses folder storage. Angka 775 memberikan hak baca, tulis, dan eksekusi kepada pemilik dan grup

chmod -R 775 /var/www/html/registration-mis/bootstrap/cache → Mengubah hak akses folder cache. Angka 775 memberikan hak baca, tulis, dan eksekusi kepada pemilik dan grup

Semua code ini mulai dari composer install sampai pada memberikan hak akses untuk folder cache, dikarenakan saya menggunakan Laravel Full Stack untuk website saya.

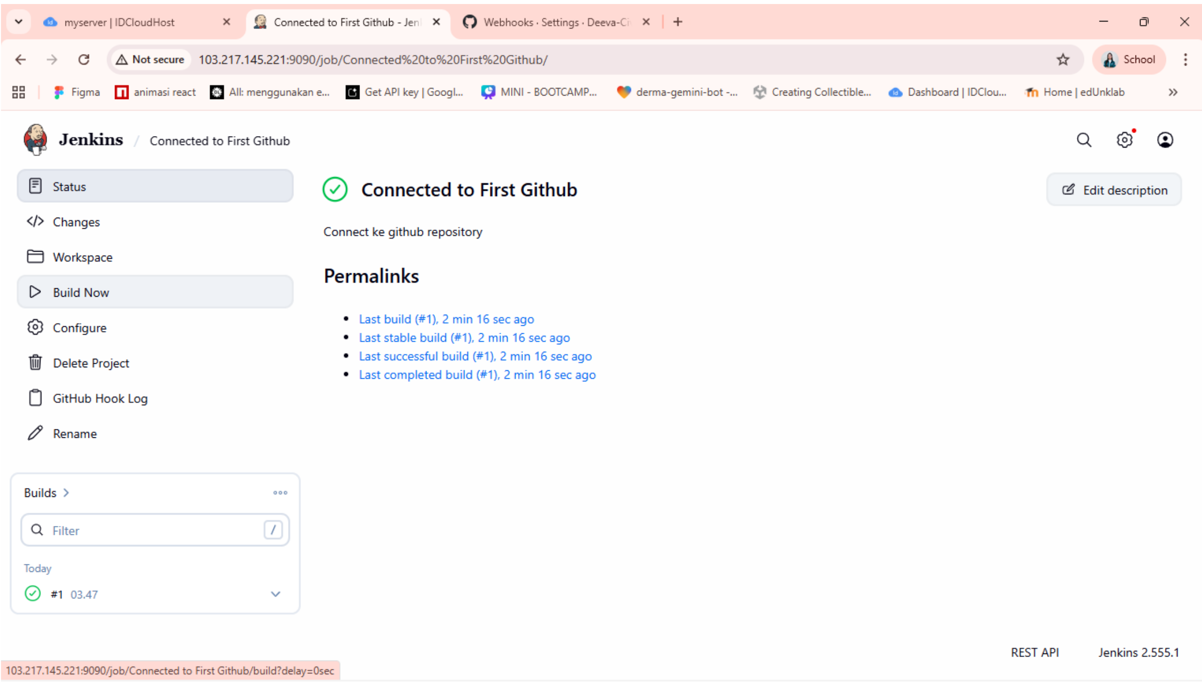
Setelah mengisi field Execute shell, klik save



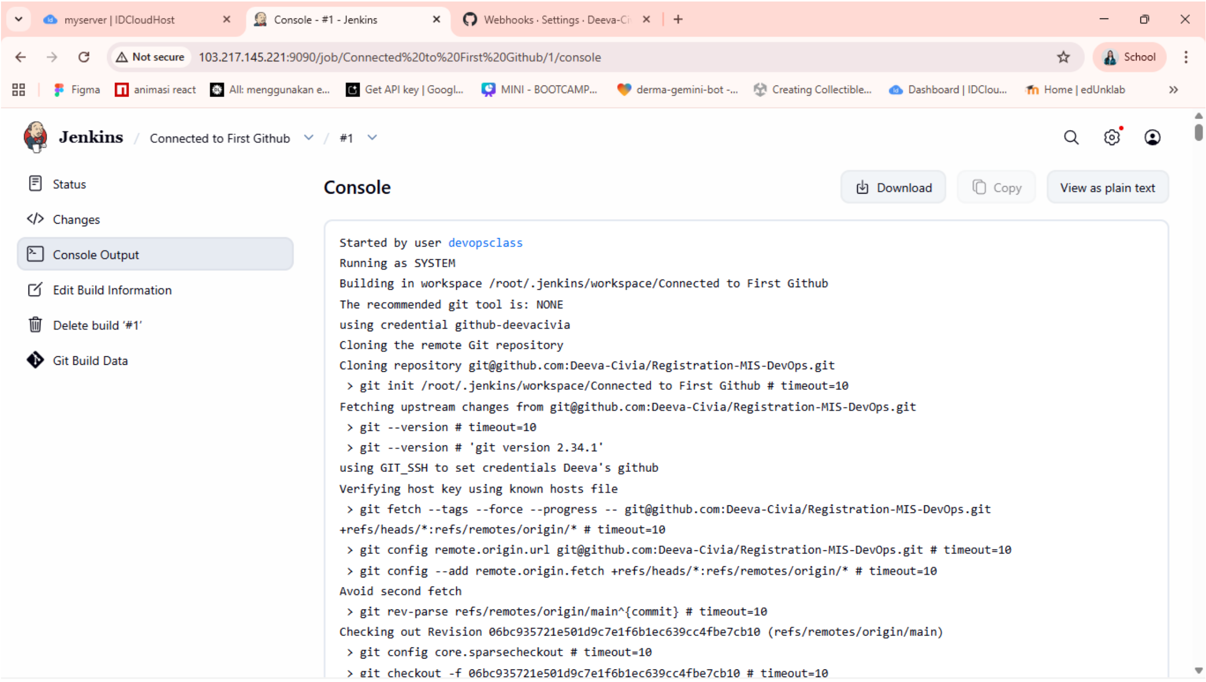
Setelah selesai melakukan konfigurasi itu, klik “Build Now” pada side bar, untuk inisiasi awal yaitu membuat workspace jenkins dan clone project.

Dan jika telah selesai, kita dapat melihat pada kolom Build terdapat “#1” berwarna hijau yang berarti proses berhasil dilakukan.

Klik “#1” tersebut, lalu buka Console Output untuk melihat detail prosesnya.

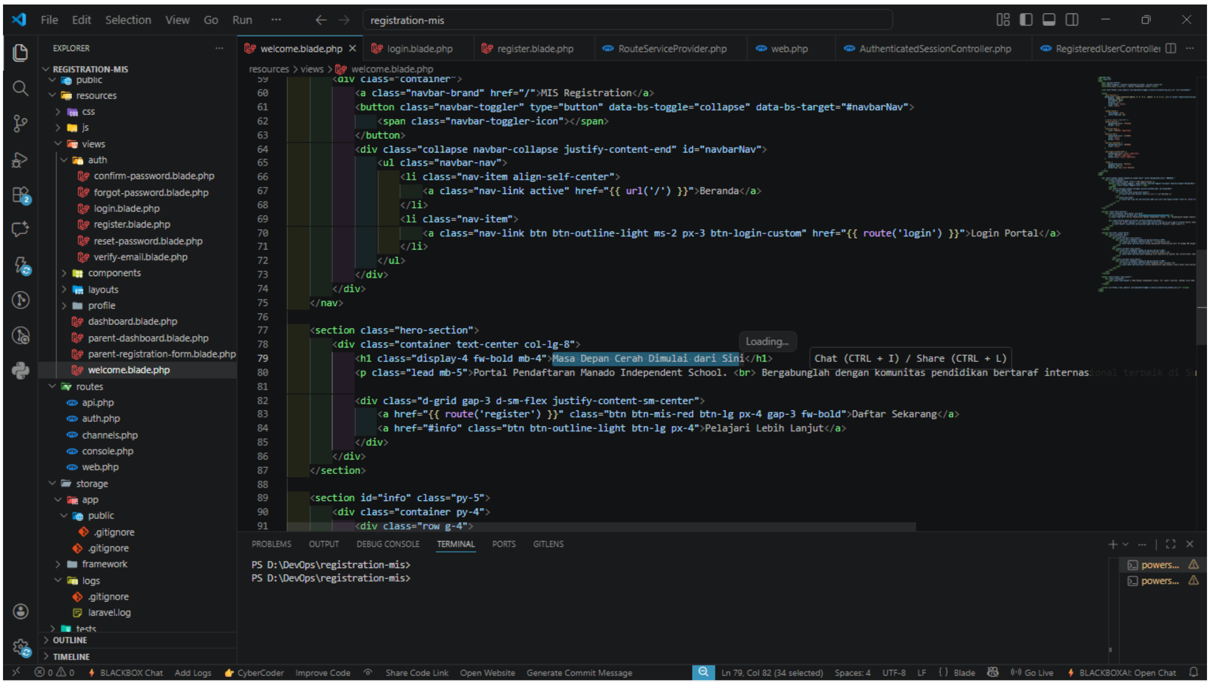


Ini isi dari console output, scroll sampai bawah dan kita akan melihat pesan Success

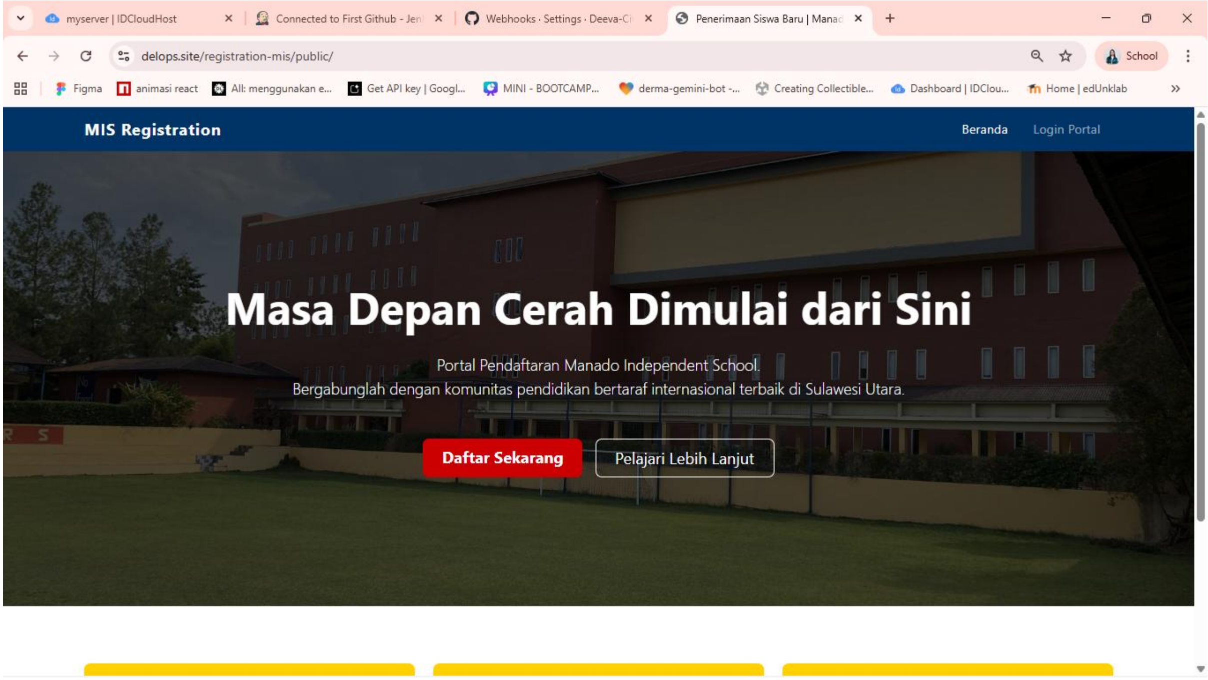


6. Pengujian CI/CD Real-time

Untuk mengetes apakah CI/CD yang telah dibuat benar berhasil, buka folder project di local melalui VS code.
ini isi dari code Landing Page website saya.

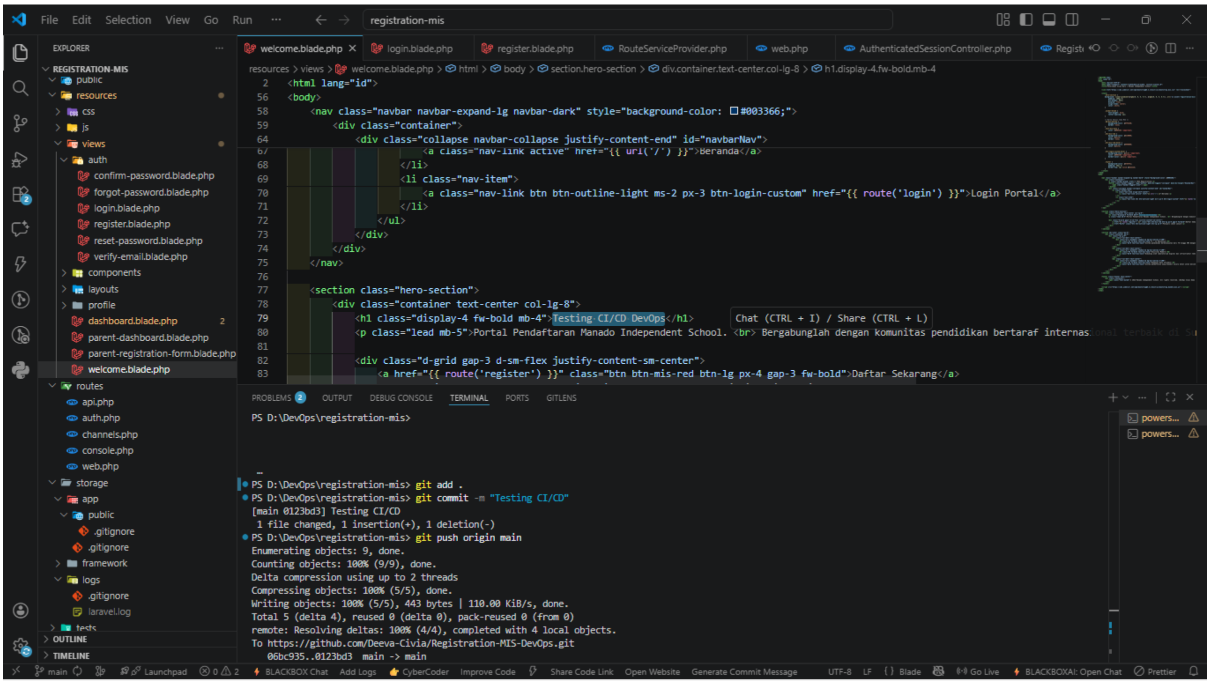


Dan ini adalah tampilan Landing Page sebelum saya melakukan perubahan

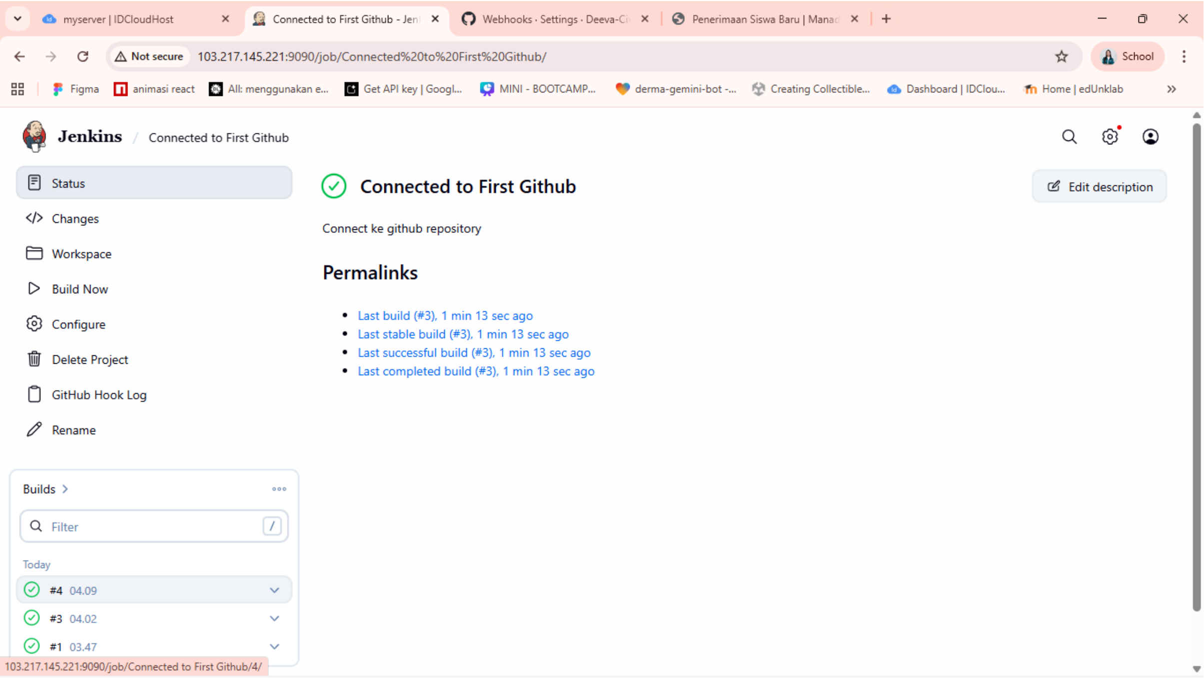


Sekarang saya akan mengubah Text “Masa Depan Cerah Dimulai dari Sini” menjadi “Testing CI/CD DevOps” melalui VS code. Lalu melakukan commit dan push ke github.

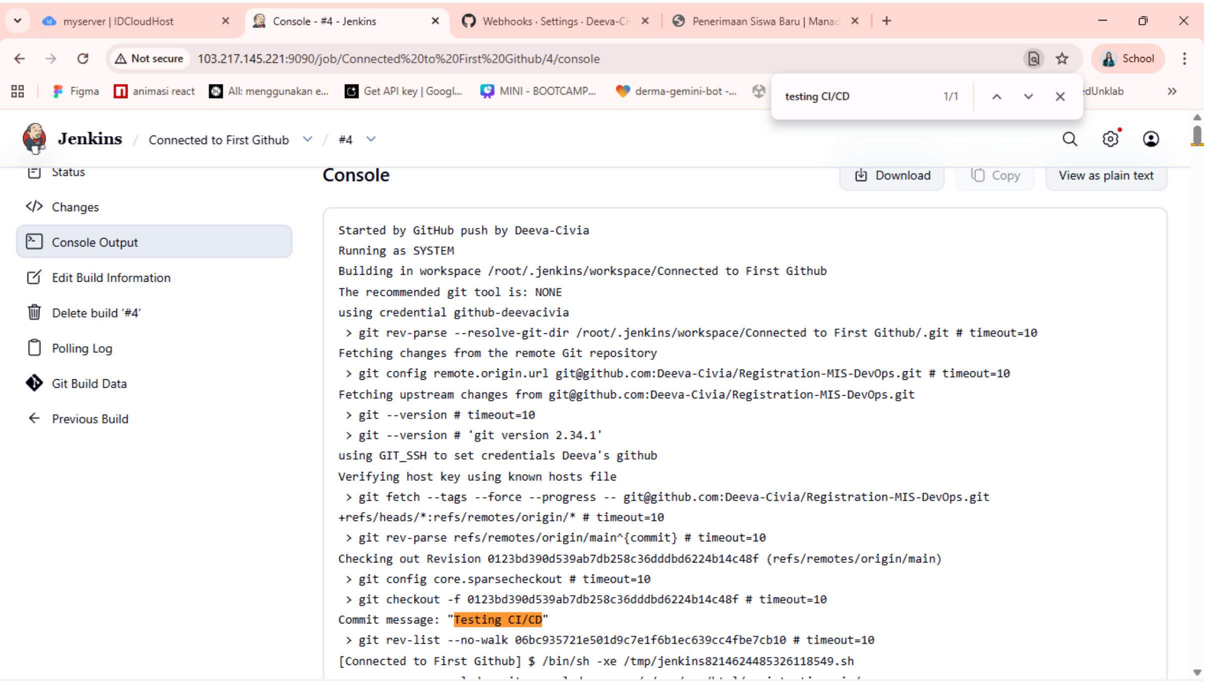
Disini saya telah selesai push.



Cek Jenkins, pada job di jenkins telah ada build terbaru “#4” dan kita akan mengecek console outputnya



Ini adalah console output dari build terbaru, dan dapat terlihat disini bahwa jenkins berhasil mengambil commit yang telah saya lakukan tadi.



Cek lagi tampilan Landing Page Website, disini tulisannya telah berhasil berubah.

Yang berarti code yang telah kita push dari local telah berhasil diambil oleh Jenkins secara otomatis.